

Research Report on

Empirical and conceptual comparisons of multi-agent vs single-agent and reasoning vs non-reasoning LLMs in software engineering

Full search query: I want to find empirical and conceptual work, restricted to software engineering tasks (such as software vulnerability detection, code generation, program repair, and related code tasks), that (1) explains why multi-agent LLM setups (two or more collaborating or debating LLM agents) do not always outperform, and can sometimes underperform, single-agent LLM approaches, and (2) compares “reasoning/thinking” LLM modes or models versus non-reasoning/instruction-following modes or models, showing for which types of software/code tasks each tends to perform better and offering explanations for these differences

Summary

Across the software engineering literature located, there is now strong empirical evidence that multi agent LLM systems and “slow/reasoning” modes are not universally superior to single agent or “fast/direct” approaches on code tasks: gains are highly task dependent, often hinge on better decomposition and tool integration rather than multi agency per se, and can be outweighed by coordination overhead, correlated hallucinations, and error propagation [1,3,5,7,8,15,16,17].

Overall Picture: What This Literature Search Shows

Key high level conclusions

- **Multi agent vs single agent:**
 - Large, carefully controlled studies show that multi agent LLM frameworks often deliver only moderate gains and can underperform well designed single agent pipelines—especially under realistic token/cost budgets [1,14,15,16,17].
 - When multi agent systems shine, it is typically on **repository scale, multi file, or design level tasks** with strong tool feedback and well engineered role decomposition (e.g., MAGIS, RefAgent, DEI committees) [9,11,12,19], but even there they are not uniformly dominant.
- **Reasoning/thinking vs non reasoning modes:**
 - Explicit comparisons show that **structured or extended reasoning** is highly beneficial for:
 - Path sensitive test generation [8],
 - Ambiguous understanding tasks such as equivalent mutant detection [3],
 - Hard repair instances when applied selectively (SIADAFIX’s “slow thinking” route) [7].
 - Conversely, **direct/instruction following or minimal pipeline modes** are competitive or superior for:
 - Many program repair tasks grounded by tests and clear error signals [3,7,15,16,17],
 - Short, pattern based code generation,
 - Situations where excessive deliberation leads to error accumulation or context bloat [1,7,16].

Overall, the literature strongly supports a **nuanced, task conditioned view** multi agent and slow/reasoning modes are powerful tools, but their benefits are tightly constrained by task structure, grounding signals (tests, compilers, analyzers), coordination design, and compute limits.

Multi Agent vs Single Agent on Software Engineering Tasks

Where multi agent systems underperform or fail to dominate

Several works explicitly ask whether complex agentic setups really beat simpler single agent or “agentless” pipelines on code tasks:

- **Cross framework, multi task evaluation**– Yin et al. [1]:
 - Evaluates seven general purpose agent frameworks on:
 - Software development, vulnerability detection, and program repair.
 - Finds only **moderate overall performance**, with **multi agent architectures often not outperforming simpler single agent pipelines**, and sometimes underperforming once cost and tokens are accounted for.
 - Diagnoses core failure modes:
 - **Coordination overhead and long trajectories** (many corrective iterations that do not yield better final solutions).
 - **Token explosion ’ context overflow ’ loss of earlier, relevant information.**

- **Inter agent hallucination and error propagation** including malformed patches or repeated wrong edits.
- Single agent systems typically keep contexts smaller, preserve history better, and control false positives more effectively [1].
- **Agentless vs agentic bug fixing on SWE Bench Lite**– Xia et al. [15,16]:
 - Agentless is a **minimal, three phase pipeline**(localization ' repair ' validation) with no autonomous planning or multi agent loops.
 - On SWE Bench Lite, it achieves the**highest reported performance among open source tools at submission time** (32% pass@1) at **much lower cost** than agentic systems [16].
 - Conclusions:
 - Complex autonomous agents are **not necessary** to achieve strong results on repository level bug fixing.
 - Much of the “benefit” of agents comes from **having any reasonable pipeline at all**, not from multi agency itself.
- **Bug fixing landscape across many tools**– Meng et al. [17]:
 - Compares multiple top performing agent and non agent methods on SWE Bench Lite/Verified.
 - Finds:
 - Instances solvable only by agents, only by non agents, and by both;**no paradigm dominates**.
 - Remaining failures attributed to:
 - Weaknesses in LLM reasoning (identifying bug relevant information, handling multiple suspicious locations),
 - Limitations in agentic flow design (poor verification strategy, inadequate exploration of patch space).
 - This reinforces that **search and feedback integration** matter more than whether the system is labeled “agentic.”
 - **Agent vs agentless on performance optimization** – SWE Perf [14]:
 - Compares a pipeline (Agentless style) and an agentic framework (OpenHands) using the**same LLM** (Claude 3.7 Sonnet) on performance optimization across real repositories.
 - Shows a **large gap to human experts for both**, with neither paradigm clearly superior across the board.
 - Highlights trade offs between:
 - Agentic **multi step interactions**(potentially better exploration),
 - Versus **simpler pipelines** (better efficiency and controllability).
 - **MAS under knowledge conflicts** – Ju et al. [23]:
 - Shows that multi agent disagreements can sometimes be self correcting on programming tasks (multiple valid solution paths), but also reveals **intrinsic limits**:
 - When task critical incorrect knowledge is injected, success rates drop, especially on single path reasoning tasks.
 - Implies that MAS robustness is **task structure dependent**and does not guarantee superiority over single agents.

Taken together, these works undermine any assumption that simply “adding agents” will improve code task performance. Instead, they show:

- Under realistic token/latency budgets, multi agent systems often:
 - Spend budget on coordination rather than on exploring more solution variants.
 - Suffer from context truncation and error cascades.
 - Fail to consistently outperform strong single agent pipelines.

Mechanisms explaining multi agent underperformance

Across empirical and conceptual work, several recurring mechanisms explain why multi agent systems underperform or offer limited gains:

- **Coordination and communication overhead**:
 - Longer dialogues and handoffs drive up token usage and latency without guaranteed accuracy gains [1,13,16].
 - Process model studies (Agile vs Waterfall vs V Model) show higher quality outputs at the cost of more coordination, especially in Agile style MAS [13].
- **Context explosion and information loss**:
 - Inter agent communication can quickly exceed context windows, causing truncation of earlier, critical details (e.g., test failures, prior decisions) [1].
 - This is particularly harmful in multi step program repair or refactoring where global consistency matters.
- **Correlated hallucinations and error propagation**:

- Agents built on the same base model and similar prompts share priors and often **converge on similar mistakes** rather than correcting each other [1,2].
- Pipeline style multi agent flows (strategy ' coding ' revision) frequently propagate early misinterpretations; downstream agents rarely overturn upstream errors [2,10,21,22].
- **Edge level error cascades at handoffs**
 - AgentAsk [5] shows that **small inaccuracies at message boundaries** (partial misinterpretations, incomplete summaries) can snowball into large failures.
 - Learned "ask" policies between agents (deciding when/what/whom to question) improve robustness, demonstrating that **communication itself must be actively managed**.
- **Cognitive deadlocks and rigid workflows:**
 - Failure analysis on SWE Bench Verified shows agentic systems can get stuck in **unproductive loops**, repeating the same failing actions or misinterpreting feedback without revising their plan [4].
 - Pipeline systems, in contrast, often fail because their workflows are **too rigid** and cannot adapt to unexpected feedback, but they avoid some agent style deadlocks [4].
- **Shared repository state drift ("of sync" code)**
 - Surveys note that when multiple agents update a shared codebase, they can operate on outdated or inconsistent views, introducing cross file inconsistencies and broken invariants [2].
 - This is a concrete risk in multi file refactoring and large scale maintenance tasks.

These mechanisms align closely with your background description: multi agent systems often amplify shared biases and add new error channels (handoffs, coordination, state drift), while consuming more compute.

When and why multi agent systems help

Despite the above, several works demonstrate that **multi agent architectures can provide substantial benefits** when carefully designed and properly grounded:

- **Repository level issue resolution and refactoring**
 - MAGIS [9] uses four specialized agents (Manager, Repository Custodian, Developer, QA) and reports significant gains over direct GPT 3.5/4/Claude 2 on SWE Bench, largely by addressing:
 - Repository scale context limitations,
 - Coordination of reading, editing, and testing across files.
 - RefAgent [12] shows that multi agent refactoring (planner, executor, tester, refiner) can outperform:
 - Single agent refactoring,
 - A search based tool,
 - And in some aspects, developer refactorings, especially in identifying opportunities and preserving behavior.
- **Committee of agents and diversity**
 - DEI [19] orchestrates multiple existing SWE agents and raises the best individual success rate on SWE Bench Lite from 27.3% to 34.3% (and up to 55% with larger committees).
 - This supports the idea that **agent diversity**, not just multiple copies of the same agent, can improve coverage.
- **Specialized MAS for code generation:**
 - AgentCoder [21] (programmer / test designer / executor) and self collaboration [22] report substantial pass@1 gains on HumanEval/MBPP vs simple direct generation, by tightly coupling generation with test design and iterative repair.
 - While compute matched comparisons are missing, these works show that **well structured MAS plus tool feedback** can offer strong performance on unit test driven benchmarks.
- **Collaborative architectures to fix agentic deadlocks:**
 - Liu et al. [4] propose an **Expert-Executor** MAS that supervises and guides an executor agent, resolving ~22% of issues previously unsolved by OpenHands on a curated subset.
 - This suggests the value of **hierarchical supervision** within MAS, where a meta agent monitors and corrects lower level loops.

The common pattern in MAS success stories is that:

- They exploit **complementary roles or views** (e.g., separate access to logs, tests, repository context),
- They use **strong external tools** (compilers, tests, static analyzers) for grounding,
- They carefully engineer **coordination structures** (process models, supervisors, committees).

By contrast, MAS with superficial role labels, no differentiated inputs, or weak grounding tend to exhibit the failure modes above.

Reasoning/Thinking vs Direct/Instruction Following Modes on Code Tasks

Only a subset of the retrieved literature explicitly isolates “reasoning” vs “non reasoning” modes (e.g., CoT vs direct, slow vs fast thinking) on the same or similar models. However, those that do provide a coherent picture of when reasoning helps and when it hurts.

Cases where explicit reasoning clearly helps

- **Coverage guided test generation**– SymPrompt [8]:
 - Introduces **code aware, multi stage prompting** that decomposes test generation into path aligned prompts based on structural analysis (using TreeSitter).
 - On Python test generation, SymPrompt yields:
 - Up to **5× more correct tests** and **+26% relative coverage** with CodeGen2, and
 - More than **2× coverage gains** for GPT 4, compared with simple one shot prompting [8].
 - Explanation:
 - This is a **canonical case where explicit reasoning is crucial**:
 - Tasks require exploring multiple execution paths, understanding data/control dependencies, and systematically covering edge cases.
 - Multi stage reasoning forces the model to consider path structure explicitly rather than relying on pattern recall.
- **Equivalent mutant detection (EMD)** – Self negotiation [3]:
 - Self negotiation is a multi agent debate among copies of the same model.
 - On EMD (a functionality understanding and code classification task), self negotiation yielded **dramatic F1 improvements** (114–351% for most models) relative to single model inference [3].
 - Explanation:
 - The task is **semantically ambiguous and interpretation heavy**, multiple perspectives and arguments help disambiguate whether mutants are equivalent.
 - This supports the idea that **reasoning/debate is especially valuable for high ambiguity understanding tasks**
- **Complex or hard repair instances** – SIADAFIX [7]:
 - SIADAFIX organizes repair into three modes (easy, middle, hard), with:
 - “Fast thinking” for simple cases (single or hard coded calls),
 - “Slow thinking” (autonomous multi round, test time scaling) for harder ones.
 - It achieves **60.67% pass@1 on SWE Bench Lite with Claude 4 Sonnet**, claimed as state of the art among open source methods at publication [7].
 - Explanation:
 - Slow, deliberate reasoning is **reserved for complex issues** where multi step diagnosis and patching are necessary.
 - This selective use of reasoning avoids its pitfalls on simpler cases.

These cases exemplify a consistent pattern: **Reasoning modes yield substantial gains when tasks require multi step, non local, or ambiguity resolving reasoning that is not easily supplied by external tests alone.**

Cases where reasoning adds little or can hurt

- **Automated Program Repair (APR)** – Self negotiation study [3]:
 - For APR tasks, self negotiation yields **comparable performance** to single LLM baselines, unlike the large gains seen on EMD [3].
 - Interpretation:
 - APR is typically **strongly grounded in external tests** and often involves **small, local edits**.
 - The dominant bottleneck is effective search under test feedback, not additional meta reasoning.
- **Vulnerability detection (AVD)** – Self negotiation [3]:
 - Results are **mixed**; some models benefit from self negotiation, others do not.
 - Vulnerability detection combines pattern matching (common templates) with deeper data flow reasoning; extra deliberation can be redundant or even introduce confusion when the base model already recognizes the pattern.
- **General bug fixing / issue resolution** – Agentless and follow up studies [7,15,16,17]:
 -

Agentless [16] uses **minimal explicit reasoning**—just a simple three step pipeline—and still outperforms complex agentic flows on SWE Bench Lite.

- SIADAFIX [7] explicitly warns that **always on slow thinking**(autonomous iteration) can lead to **continuous error accumulation**: repeated misinterpretations and cascading wrong edits.
- Meng et al. [17] identify main obstacles as:
 - Identifying bug relevant context,
 - Localizing faults,
 - Checking patch completeness and global impact, rather than lack of explicit CoT.
- **Multi agent debate (MAD) for code summarization/translation**– Chun et al. [6]:
 - Finds that MAD can improve some instances but is **not a universal “silver bullet”**:
 - Gains are modest and uneven.
 - Debate may focus on superficial aspects (style, naming) rather than deeper semantics unless carefully constrained.
 - This mirrors the broader observation that **debate without strong external grounding does not guarantee correctness**.

Overall, for many **test driven repair and standard code generation tasks** the literature suggests that **direct, pattern based behavior plus strong external feedback often suffices**, and additional explicit reasoning can:

- Consume tokens and reduce exploration capacity,
- “Lock in” incorrect designs through overconfident step by step planning,
- Introduce new failure channels (e.g., deadlocks in agentic flows).

Emerging “fast vs slow thinking” and adaptive reasoning

Several works converge on the idea that **reasoning depth should be dynamically controlled**, not hard wired:

- **SIADAFIX [7]**:
 - Treats fast vs slow thinking as an explicit design axis and uses issue description classification to decide:
 - When a simple, direct approach is enough,
 - When to invoke heavy, agentic, multi round reasoning.
 - Empirically validates that this **adaptive strategy outperforms indiscriminate slow thinking**.
- **Agentless vs deep agentic flows [15,16]**:
 - Implicitly show that **“less is more”** for many SWE Bench Lite issues:
 - A single pass localization–repair–validate loop often does better than complex self reflective or planning heavy agents.
- **Learned inter agent clarification (AgentAsk)[5]**:
 - Instead of having agents always elaborate reasoning at each step, AgentAsk learns **when clarification is needed** at coordination edges, optimizing the trade off between:
 - Accuracy improvements and
 - Latency/cost overhead.

These designs collectively point toward **future systems that treat reasoning (and multi agency) as a resource to allocate** based on:

- Estimated task difficulty,
- Ambiguity and lack of external signals,
- Required depth of global reasoning.

Task Specific Patterns for Multi Agent and Reasoning Modes

Bringing both axes together—multi agent vs single agent and reasoning vs direct modes—the literature supports the following task level patterns (which align well with your background intuition):

1) Program repair and automated bug fixing

- **Grounded by tests; many local patches.**
- Evidence:
 - Self negotiation gives limited gains for APR [3].

- Agentless minimal pipelines outperform complex agents on SWE Bench Lite [16].
- SIADAFIX's fast thinking is preferred for simple bugs; slow thinking is reserved for complex ones [7].
- Large cross tool evaluations [1,14,17] show no consistent MAS advantage.
- Takeaway:
 - Performance is driven more by:
 - Quality of test feedback,
 - Localization accuracy,
 - Patch search strategies,
 than by multi agent collaboration or explicit chain of thought.
 - Single agent or minimal pipelines plus strong tools (tests, compilers, static analyzers) are oftenthe **best baseline**, with carefully controlled slow reasoning only where truly needed.

2) Code understanding and classification (e.g., equivalent mutant detection, some vulnerability detection)

- **High ambiguity, semantic reasoning, subtle distinctions.**
- Evidence:
 - Self negotiation yields very large gains on EMD [3].
 - Results on vulnerability detection are mixed but suggest some benefit from richer reasoning for more complex vulnerabilities [3].
- Takeaway:
 - Multi agent debate or self negotiation can be highly beneficial when tasks are **under specified and rely heavily on interpreting code behavior** without strong external execution feedback.
 - Here, **reasoning modes clearly add value**, and MAS may be competitive or superior if carefully designed.

3) Test generation and coverage oriented tasks

- **Need for path coverage and non local reasoning.**
- Evidence:
 - SymPrompt's multi stage reasoning dramatically improves coverage and correctness over direct prompting [8].
- Takeaway:
 - Structured prompting that elicits explicit reasoning about code structure (control/data flow) provides **substantial gains**.
 - This can be achieved within a **single agents** system via multi stage prompts, showing that multi agency is not a prerequisite; the key is the **reasoning structure**.

4) Code generation (functions, algorithms, small programs)

- **Pattern heavy; sometimes compositional.**
- Evidence:
 - AgentCoder [21], self collaboration [22], and similar works show that multi agent + tests can significantly boost pass@1 on HumanEval/MBPP.
 - However, compute matched direct answer baselines with similar test integration are generally absent, and later work emphasizes search + tests over agency per se.
- Takeaway:
 - For standard benchmarks where solutions resemble training patterns, **direct generation plus a bit of self correction or testing** is often enough.
 - Multi agent and extended reasoning help more on:
 - Harder, multi step, multi API composition tasks,
 - Or when integrated with strong test/verification loops.

5) Repository level development, refactoring, and performance optimization

- **Large contexts, multi file dependencies, global invariants.**
- Evidence:
 - MAGIS [9] and RefAgent [12] show multi agent benefits for repository level issue resolution and refactoring.
 - SWE Perf [14] finds both agentic and agentless methods far from human experts, but MAS offer richer exploration of multi step performance optimizations.

- Process model MAS [13] show that Agile style iterative coordination achieves higher quality (at higher cost).
- Takeaway:
 - Here, **role specialization and multi step coordination** are more often beneficial, but:
 - The key is **effective orchestration under constraints**,
 - Not simply increasing the number of agents or depth of reasoning.
 - There is still no clear evidence that MAS universally beat well designed single agent pipelines under equal budget, but they can reach higher ceilings in unconstrained settings.

Gaps and Opportunities Relative to Your Research Questions

Given your specific focus—explaining when and why multi agent setups and reasoning modes underperform or outperform single agent/direct modes in SE tasks—the retrieved literature provides a strong foundation but also reveals several open gaps:

- **Limited compute matched, mode controlled comparisons:**
 - Many MAS code generation papers (AgentCoder [21], self collaboration [22], CoopetitiveV [10]) do not:
 - Compare against **single agent methods with equivalent test integration and compute** or
 - Include **CoT vs no CoT** ablations on the same backbone.
 - This makes it hard to quantify how much improvement comes from:
 - More sampling,
 - Better tooling,
 - Versus the multi agent structure or explicit reasoning.
- **Sparse detailed error analysis for reasoning vs direct modes:**
 - Self negotiation [3], SymPrompt [8], and SIADAFIX [7] are notable exceptions; most works treat reasoning as a design feature, not an experimental variable.
 - There is room for **systematic per task error taxonomies** comparing:
 - Direct vs CoT prompts,
 - Fast vs slow modes, across code tasks (generation, repair, vulnerability detection).
- **Underexplored interactions between MAS and reasoning modes:**
 - Many MAS implicitly use reasoning (planning, debate) but rarely disentangle:
 - “Multiple agents, shallow reasoning” vs
 - “Single agent, deep reasoning” under the same budget.
 - This is precisely the comparison you flagged in your background; the current literature only partially addresses it.
- **Task structure aware evaluations are still early:**
 - Ju et al. [23] begin to connect **solution space topology** (single vs multiple valid paths) to MAS robustness.
 - Extending this to:
 - Different bug types and repair patterns,
 - Different vulnerability classes,
 - Different refactoring scopes, could provide a more principled theory of when multi agent/slow reasoning is beneficial.

These gaps point toward promising research directions tightly aligned with your goals:

- Designing experiments that:
 - Control for compute and tooling,
 - Compare single vs multi agent, fast vs slow modes, on matched tasks and models,
 - Include detailed error/failure analyses per condition.
- Developing **adaptive controllers** that:
 - Learn when to invoke multi agent debate vs single agent reasoning,
 - When to switch from fast to slow thinking,
 - Based on task features (complexity, ambiguity, availability of tests).

In summary, the literature you retrieved shows that both multi agent LLM systems and explicit reasoning modes can add significant value in software engineering, but only under specific, well understood conditions; they are far from universally superior, and carefully designed single agent pipelines with judicious, task aware reasoning often match or outperform more complex multi agent or heavily deliberative setups [1,3,7,8,15,16,17].

Categories

Comparative Landscape: Key Dimensions Across the Papers

To help navigate this literature specifically from the perspective of:

- (1) multi agent vs single agent LLM setups on software/code tasks, and
- (2) “reasoning/thinking” vs non reasoning/instruction following modes,

the tables below compare the most relevant aspects across the retrieved papers, followed by short comparative highlights.

Multi Agent vs Single Agent Comparisons on Software Engineering Tasks

This table focuses on whether each work empirically contrasts multi agent (or agentic) systems with single agent (or “agentless”) systems on SE tasks, what tasks they study, and what they conclude about relative performance and failure modes.

Table 1. Empirical comparisons of multi agent vs single agent systems on SE tasks

Ref	Main SE task(s)	System types compared	High level performance outcome	Key underperformance / advantage explanations
[1]	Software development, vulnerability detection, program repair	Seven general purpose multi agent frameworks vs single agent pipelines	Multi agent frameworks show only moderate performance; some multi agent systems underperform simpler single agent pipelines in effectiveness and cost	Multi agent architectures incur coordination overhead, longer trajectories, and many correction attempts; token growth causes information overload and context truncation; inter agent hallucinations propagate identical errors; malformed patches arise from agents lacking specialized tools; single agent systems keep contexts smaller and preserve history better [1]
[3]	Equivalent mutant detection (EMD), automated vulnerability detection (AVD), automated program repair (APR)	LLM “self negotiation” (multi agent debate among same model) vs single LLM	Large gains on EMD; mixed results on AVD; APR performance comparable to single LLMs [3]	Self negotiation helps resolve ambiguity in understanding/code classification (EMD) but does not consistently help patch generation (APR) or vulnerability detection; improvements are task dependent, indicating that debate does not universally dominate [3]
[4]	Automated issue solving (SWE Bench Verified)	Multiple agentic tools vs pipeline style / simpler baselines; new “Collaborative Expert–Executor” vs OpenHands (agent) and proprietary models	Collaborative Expert–Executor resolves a subset of issues that strong agentic systems failed, with performance comparable to strong proprietary single agent systems on a curated subset [4]	Detailed failure taxonomy: most agentic failures arise from flawed reasoning and “cognitive deadlocks” in iterative workflows; rigid workflows and coordination limits lead to getting stuck in unproductive loops; collaborative architecture reduces some of these issues [4]
[9]		MAGIS multi agent framework vs direct single in-	MAGIS significantly outperforms direct GPT 4 and others on SWE Bench (e.g., 13.94% issues resolved,	MAGIS addresses repository scale context constraints via role specialization (Manager, Custodian, Developer, QA) and structured collaboration; failure analysis attributes single LLM failures to context

	GitHub issue resolution (SWE Bench)	stance LLMs (GPT 3.5, GPT 4, Claude 2)	~8× improvement over direct GPT 4) [9]	limits and missing repository level reasoning; multi-agent gains are tied to better decomposition and tool usage, rather than “multi agent” per se [9]
[11]	Bug detection and code refactoring	Cooperative multi agent architecture (static analysis, test generation, root-cause, refactoring agents) vs single agent and non LLM baselines	Reported higher fix precision and refactoring quality than single agent baselines, especially on multi file defects and design smells [11]	Gains attributed to combining LLM reasoning with static analysis and structured coordination via a planning agent; failure modes include spec ambiguities, over refactoring, and flaky tests rather than explicit multi agent pathologies [11]
[12]	Automated Java refactoring	RefAgent multi agent vs (1) single agent LLM refactoring, (2) search based refactoring tool, (3) developer refactorings	RefAgent reports improved ability to identify refactoring opportunities and preserve behavior compared to single LLM and search based tools, with competitive test-passing rates [12]	Authors argue role specialization (planning, execution, testing, refinement) plus tool integration mitigate hallucinations and preserve behavior; ablations show each agent’s contribution, but the provided excerpts do not highlight systematic underperformance cases [12]
[13]	End to end software generation (11 projects)	Multiple multi agent teams organized via Waterfall, V-Model, Agile; no explicit single agent baseline in the excerpt	Agile multi agent processes achieve highest code quality but at higher computational cost; Waterfall uses fewer tokens but lower quality [13]	Process model shapes coordination: Agile’s iterative feedback reduces failure rates; static processes (Waterfall, V Model) miss early bug resolution; coordination overhead is substantial, but no direct single agent comparison is reported in the excerpt [13]
[14]	Repository level performance optimization (SWE Perf benchmark)	“Agentless” pipeline vs OpenHands agent framework (both using Claude 3.7 Sonnet) + other methods	There is a large gap to expert performance; comparison between Agentless and OpenHands is reported (details in figures/tables), showing differing strengths across file level vs repo level approaches [14]	Authors note that agent-based OpenHands enables iterative reasoning and multi step interaction, while Agentless uses a hierarchical pipeline for localization, repair, and validation; the balance between search depth, tool use, and overhead is central to performance differences [14]
[15]/[16]	Repository level bug fixing (SWE Bench Lite and SWE Bench Lite S)	Agentless minimal pipeline vs existing open source agent based systems	Agentless attains highest performance (32.0% on SWE Bench Lite) at lowest cost (~\$0.70) among open source software agents at submission time [16]	Shows that simple three-phase localization’repair’-validation without agentic decision making can outperform complex agents; argues that complexity and long trajectories of agents introduce overhead without commensurate gains; also indicates many agent failures are due to brittle flows and poor tool use rather than lack of “agency” [16]
[17]	Automated bug fixing (SWE Bench Lite/Verified)	Multiple top performing agent and non agent systems (some proprietary)	Some instances solvable only by agents, some only by non agents; no paradigm dominates; overall capability gap remains large [17]	Failures attributed to (a) LLM limitations in identifying bug relevant info, (b) difficulty handling multiple suspicious locations, (c) incomplete patch verification and global impact reasoning, and (d) randomness/diversity management; weaknesses arise both from LLM reasoning and agentic flow design [17]
[19]			DEI improves best individual agent resolve rate (e.g., from 27.3% to 34.3%; best	Gains come from leveraging diversity across agents; paper does not deeply analyze when commit-

	GitHub issue resolution (SWE Bench Lite)	DEI meta framework combining multiple SWE agents vs best individual agent	committee up to 55% on SWE Bench Lite [19]	tees underperform or compare against tuned single agent pipelines under equal compute [19]
[20]	Software development from NL descriptions (NLDD dataset)	Experiential Co Learning multi agent (instructor + assistant) vs single agent GPT Engineer and other MAS (MetaGPT, ChatDev)	Co Learning outperforms GPT Engineer on completeness/executability and is more time efficient than other MAS, but slower than single agent [20]	Authors credit better task decomposition and experience reuse for quality gains; they acknowledge runtime overhead vs single agent and only slight improvements in consistency [20]
[21]	Code generation (HumanEval, MBPP)	AgentCoder three agent system vs single agent baselines	Reports large pass@1 improvements (e.g., 87.8% vs 61.0% on HumanEval; 89.9% vs 51.8% on MBPP) [21]	Improvements attributed to separation of programmer, test designer, and executor roles plus iterative testing; compute budget parity vs single agent not detailed in excerpt, so gains may reflect more search/feedback, not multi agency alone [21]
[22]	Code generation (multiple benchmarks)	Self collaboration multi agent role play vs direct/base generation	Reports 29.9–47.1% relative pass@1 improvements and often surpasses GPT 4 [22]	Gains attributed to role differentiation (analyst, coder, tester) and interaction protocols; again, no explicit compute matched single agent baseline is described in the excerpt [22]
[23]	Programming (HumanEval, GAIA) + reasoning tasks (CounterFact, MQuAKE cf)	Multi agent collaborative settings under injected disagreements; single agent baselines not emphasized in provided text	Shows that heterogeneous knowledge can help or harm depending on task structure; programming tasks show more self repair capability than single path reasoning tasks [23]	When agents carry task - critical incorrect knowledge, success plummets on single path reasoning but is less impacted on programming tasks; MAS often bypass incorrect facts in programming, revealing both robustness and intrinsic limits [23]

Explanatory Mechanisms: Why Multi Agent Setups Fail to Dominate

Across these works, experts will care not just about “who wins” but about the mechanisms. The table below focuses on the most explicit failure explanations and design level insights related to multi agent underperformance or mixed results.

Table 2. Prominent explanations/mechanisms for multi agent underperformance or mixed gains

Ref(s)	Mechanism / explanation (SE specific)	Evidence / context
[1]	Coordination overhead and trajectory length	Multi agent frameworks incur longer trajectories and many correction attempts, raising cost and often not improving final success vs single agent pipelines [1]
[1]	Information overload and context loss	Inter agent communication grows token counts, exceeding context length and causing loss of important historical information, hurting repair and development [1]
[1,2]	Inter agent hallucination and error propagation	Agents share similar priors and prompts; hallucinated mistakes in early messages propagate, leading to consistent but wrong patches or detections [1,2]
[1]	Tool misuse and format errors	Multi agent repair frameworks sometimes generate malformed diffs or patches because specialized tooling is not consistently used by all agents; specialized single agent tools avoid such format failures [1]
[2]	“Of sync” states in shared codebases	Multi agent pipelines updating a shared repo can drift out of sync; agents operate on stale views, introducing inconsistencies and broken invariants across files [2]
[2,10,21,22]	Pipeline style error propagation	In sequential multi agent pipelines, a flawed early “strategy” or analysis is handed off and

		rarely overturned downstream, leading to systemic error propagation [2,10,21,22]
[4]	Cognitive deadlocks and rigid workflows	Agentic systems on SWE Bench Verified often get stuck in loops (repeating failing actions) or rigidly follow workflows that do not adapt to unexpected feedback [4]
[4]	Architecture specific failure fingerprints	Failure taxonomy shows distinct patterns for agentic vs pipeline tools; many agentic failures are tied to reasoning and iteration anomalies (e.g., misinterpreting test failures, mismanaging iterations) [4]
[5]	Edge level error cascades at inter agent hand offs	Small inaccuracies (misinterpreted instructions, partially wrong summaries) at message boundaries between agents propagate and magnify; AgentAsk adds targeted “clarification questions” at these edges and improves MAS robustness on coding tasks [5]
[13]	Process model–induced inefficiencies	Different multi agent software process models (Waterfall, V Model, Agile) lead to different failure/cost profiles; static models may miss early bug resolution and over generate documentation or test artifacts, increasing cost without quality gains [13]
[16,15]	Unnecessary complexity vs minimal pipelines	Agentless shows that a simple, fixed pipeline with clear stages (localization, repair, validation) can outperform complex autonomous agents; authors argue agent designs often add complexity and tool orchestration without proportionate accuracy gains [16]
[17]	Limited reasoning and patch search abilities in both agents and non agents	Comprehensive evaluation shows no approach dominates; failures stem from poor identification of bug relevant info, inability to handle multiple candidate locations, and inadequate patch completeness checking, suggesting that improved reasoning and search strategies may matter more than multi agency per se [17]
[18]	Current LMA systems limited by both agent synergy and base model capabilities	Survey and case studies note that agent collaboration can improve robustness but is constrained by individual agent weaknesses, tooling, and orchestration; calls for optimizing agent synergies and stronger correctness signals [18]
[23]	Task structure–dependent robustness to disagreements	Programming tasks (multiple valid solution paths) allow MAS to “bypass” incorrect knowledge injected into some agents more often than single path reasoning tasks, implying that MAS robustness is shape of solution space dependent [23]

These mechanisms collectively reinforce that:

- Multi agent systems can add value, especially on complex repository level tasks with strong tools and good decomposition [9,11,12,19,21,22], but
- Under fixed or realistic budgets, coordination overhead, correlated hallucinations, and error propagation often erode or reverse advantages compared to well designed single agent pipelines [1,4,15,16,17].

Reasoning/“Thinking” Modes vs Non Reasoning Modes for Code Tasks

Only a subset of papers explicitly contrast “reasoning/thinking” modes (longer deliberation, chain of thought, slow thinking, self negotiation) against more direct instruction following or “fast” modes on software tasks.

Table 3. Comparisons between reasoning/thinking modes and non reasoning modes on SE tasks

Ref	Task(s)	Reasoning vs non reasoning conditions	Main empirical outcome	Explanations / takeaways
[3]	EMD, AVD, APR	“Self negotiation” multi agent debate (explicit iter-	Huge gains for EMD (114.72–351.01% improvements in F1 for most models), comparable per-	Debate style reasoning is particularly beneficial for functionality understanding (EMD) where ambiguity is high; for APR, where external tests guide correctness, extra reasoning provides limited gains; vulnerability detection shows

		ative reasoning) vs direct single LLM inference	formance for APR, mixed results for AVD [3]	model and domain dependent behavior [3]
[7]	Program repair (SWE Bench Lite)	SIADAFIX combines “slow thinking” bug fix agent (complex iterative repair, test time scaling) with “fast thinking” components that classify/route issues; adaptively chooses easy/middle/hard repair modes	Achieves 60.67% pass@1 with Claude 4 Sonnet, reported as state of the art among open source methods on SWE Bench Lite [7]	“Fast thinking” (single/hard coded calls) is used for simple bugs; “slow thinking” (autonomous multi round planning) is invoked for complex ones; authors argue that indiscriminate slow/autonomous repair accumulates errors, and that adaptive selection mitigates continuous error accumulation [7]
[8]	Coverage guided test generation	SymPrompt multi stage, path aligned prompting (elicits multi step code aware reasoning) vs standard one shot prompting on same model	SymPrompt yields ~5× more correct tests and +26% relative coverage for CodeGen2; >2× coverage improvement for GPT 4 vs baseline prompting [8]	Code aware, stepwise reasoning particularly helps explore execution paths and handle non local dependencies in tests; authors show that structured reasoning prompts can substantially outperform naive prompting without changing the base model [8]
[1]	Development, vulnerability detection, program repair	Reflective reasoning vs more direct approaches within agent frameworks (e.g., OpenHands vs others)	OpenHands shows stronger reflective reasoning; performance and cost trade offs differ across tasks [1]	Reflective reasoning helps some tasks but increases token usage; multi step reflection must be balanced against context limits and cost [1]
[4]	Issue solving	Agentic tools with iterative reasoning vs simpler/pipeline approaches; Expert–Executor collaborative reasoning vs prior OpenHands runs	Collaborative reasoning reduces some categories of reasoning failures (e.g., bug reproduction, iteration anomalies) on previously failed issues [4]	Shows that structured, supervised reasoning (Expert–Executor) can fix deadlocks seen in autonomous agents; suggests that unconstrained “slow thinking” is not always beneficial without higher level control [4]
[7,15,16]	Program repair / issue resolution	Fast vs slow thinking (SIADAFIX) [7]; minimalist “non agentic” reasoning (Agentless) vs complex autonomous agents [15,16]	SIADAFIX and Agentless both show that careful control of reasoning depth can outperform always on agentic search [7,16]	Works collectively argue that more thinking/steps is not always better; shallow, well targeted reasoning combined with strong tests/tooling can be superior to deep autonomous deliberation that accumulates errors [7,16]

Note that many multi agent code generation systems (e.g., AgentCoder [21], Self Collaboration [22], CoopetitiveV [10]) use chain of thought–like multi step reasoning internally, but the provided excerpts do not describe explicit ablations comparing “with vs without CoT” or “deliberate vs direct” modes on the same base model under controlled budgets. Instead, they report overall gains over simple baselines.

Surveys and Conceptual Work: Integrating Both Axes

Two survey/vision papers synthesize many of these results in broader frameworks that are directly relevant to your questions:

Table 4. Survey and meta analytical contributions

Ref	Scope	Contribution to multi agent vs single agent debate	Contribution to reasoning vs non reasoning debate
[2]	Survey on LLM based agents for code generation	Categorizes single agent vs multi agent architectures, documents design patterns and reported gains; notes multi agent failure modes such as “of sync” code states and hallucination propagation, and highlights that many reported gains are tied to tool integration and decomposition rather than multi agency alone [2]	Discusses “thinking longer” and extended reasoning modes as a way for smaller agents to catch up; notes that deep domain specific reasoning remains limited, especially without strong external checks [2]
[18]			

	Systematic review of LLM based MAS for SE	Maps LMA applications across the SDLC, presents case studies showing both effectiveness and limitations, and calls for optimizing agent synergy and individual capabilities; emphasizes that collaboration aims to improve robustness but is constrained by LLM weaknesses and orchestration quality [18]	Frames collaborative reasoning (debating, validating, examining roles) as a way to enhance factuality and reasoning but acknowledges that evidence of consistent superiority is limited and that stronger correctness signals and better synergy design are needed [18]
--	---	---	---

Cross Paper Comparative Highlights

Putting these tables together, key comparative insights an expert might care about are:

- When multi agent underperforms or equals single agent:
 - Large scale empirical work on multiple SE tasks and benchmarks shows that multi agent systems do not reliably dominate:
 - Agentless single pipeline approaches can outperform complex agents on SWE Bench Lite at lower cost [16].
 - In empirical studies of multiple frameworks, multi agent systems incur overhead, propagate errors, and sometimes fall behind simpler pipelines [1,4,17].
 - Debate/self negotiation improves some classification/understanding tasks but not APR, where tests provide strong external guidance [3].
 - Under fixed or realistic budgets, extra coordination and reasoning can reduce the number of distinct candidate patches or solutions explored, which is critical for pass@k metrics.
- When multi agent can excel:
 - On repository level issue resolution and refactoring, where:
 - Role specialization plus strong tool integration addresses context length and repository scale reasoning challenges [9,11,12,19].
 - Committees of diverse agents can raise ceilings when compute is not the primary constraint [19].
 - On code generation benchmarks where multi agent systems effectively couple generation with test design and execution [21,22], although compute matched baselines are often missing.
- Reasoning vs non reasoning:
 - Explicit comparative work shows:
 - Structured, code aware multi stage reasoning can significantly improve test generation quality and coverage [8].
 - Adaptive “fast vs slow thinking” in repair (SIADAFIX) and minimalist pipelines (Agentless) indicate that:
 - Always on deep reasoning/agentic loops can cause error accumulation.
 - Controlled or targeted reasoning, combined with robust external tests, is often more effective [7,16].
 - Debate style reasoning especially helps ambiguous understanding tasks (EMD) but brings limited gains to tasks already grounded by strong test feedback (APR) [3].
 - Unique conceptual contributions about failure mechanisms:
 - Edge level error cascades and the value of inter agent clarification questions [5].
 - Cognitive deadlocks and rigid workflows in agentic systems [4].
 - Repository state drift (“of sync” multi agent codebases) and pipeline error propagation [2,10].
 - Task structure–dependent robustness to disagreements (programming vs single path reasoning) [23].

These converging findings are central to your overarching research question: they collectively indicate that neither multi agent architecture nor explicit “reasoning modes” are universally superior. Their benefits are strongly mediated by:

- task type (understanding vs generation vs repair),
- presence/strength of external correctness signals (tests, compilers, analyzers),
- coordination/orchestration quality, and
- compute/token budgets and context limits.

Timeline

Early Multi Agent Optimism in Code Tasks (H2023)

Initial framing: “LLM teams” as the path to stronger software agents

The first wave of work in this space largely **assumed** that multi agent setups would significantly outperform single LLMs on software tasks, with limited scrutiny of cost or failure modes:

- **Role based “virtual software teams.”**
 - **Self Collaboration via ChatGPT[22]** (ChatGPT split into analyst / coder / tester) is a canonical early system:
 - Shows large Pass@1 gains (H30–47% relative) over direct code generation on multiple benchmarks.
 - Motivates *role specialization + iterative collaboration* as a software engineering analogue of human teams.
 - Emphasis is on *success cases*; there is little analysis of where/why multi agent fails or underperforms.
- **Multi agent with experience reuse.**
 - **Experiential Co Learning[20]** (instructor + assistant) adds persistent “experience” memory on top of a multi agent decomposition.
 - Outperforms the single agent GPT Engineer baseline in completeness and executability, but is *lower than single agent* while still faster than some other MAS [20].
 - This is one of the first hints of a **time/efficiency penalty** for multi agent setups, but it is treated as acceptable overhead rather than a central research question.
- **Agents for code generation with testing loops.**
 - **AgentCoder [21]** (programmer / test designer / test executor) demonstrates very large gains on HumanEval/MBPP compared to single agent baselines, using auto generated tests and iterative repair.
 - The narrative is strongly **pro multi agent** attributing improvements to “necessity of multiple agents” and multi step reasoning [21].
 - There is no compute matched comparison or detailed error analysis, reinforcing the early “**multi agent as silver bullet**” perception.

Overall, the **2023–early 2024** period is characterized by:

- **Multi agent decomposition as the default** for “serious” SE tasks.
- **Human process analogies** (software teams, instructor/assistant) as the main conceptual frame.
- Limited attention to:
 - Coordination overhead.
 - Failure taxonomies.
 - Comparisons to well designed single agent pipelines under fixed budgets.

2024: Consolidation, Large Scale Benchmarks, and First Backlash

Repository level benchmarks and practical agents

By 2024, the community shifts from small, synthetic tasks to **repository scale, test driven benchmarks** and this exposes the limitations of complex agentic setups:

- **SWE Bench Lite and SWE Bench Verified as central testbeds.**
 - **MAGIS [9]**: a four agent system (Manager, Repository Custodian, Developer, QA Engineer) for issue resolution.
 - Shows sizable gains over *direct* GPT 3.5/4/Claude 2 usage [9], reinforcing the value of better orchestration and tool use.
 - Still largely success oriented; underperformance is attributed to context limits and generic failure factors.
 - **Empirical Study on LLM based Agents for Automated Bug Fixing[17]**:
 - Compares multiple agentic and non agent systems on SWE Bench Lite/Verified.
 - Attributes remaining failures to **LLM reasoning limits and agentic flow design**, highlighting that **better flow and reasoning**—not just more agents—are needed [17].
- **Committee of agents and diversity.**
 - **DEI (Diversity Empowers Intelligence) [19]**: orchestration of multiple existing SWE agents into a “committee.”
 - Improves the best individual open source agent from 27.3% to 34.3% on SWE Bench Lite, further up to 55% with larger groups [19].
 - Emphasizes **agent diversity** as the driver of gains, but does not deeply examine cases where committees fail or cost/latency trade offs at fixed budgets.

The Agentless turning point: “Do we really need complex agents?”

The strongest 2024 inflection point is **Agentless**:

- **Agentless: Demystifying LLM based SE Agents**[16] (arXiv 2024; journal version [15] in 2025):
 - Proposes a simple, **three phase pipeline**(localization ' repair ' patch validation) with**no autonomous planning** or agent loops.
 - On SWE Bench Lite,**Agentless surpasses all open source agentic systems**at the time, both in:
 - **Effectiveness** (32.0% resolve rate, 96 fixes) and
 - **Cost** (~\$0.70) [16].
 - This directly challenges the implicit assumption that **agentic complexity is necessary** for strong performance on end to end SE tasks.

This paper is widely cited and becomes a **conceptual pivot**:

- Subsequent works (e.g., [1,4,14,18,19]) **benchmark against Agentless** or explicitly discuss “agentless vs agents,” framing the debate as:
 - **Structured single agent pipelines vs multi agent autonomy**rather than “agents vs plain completions.”

Surveys and conceptual consolidation

- **LLM Based Multi Agent Systems for Software Engineering**[18] (ACM TOSEM 2024) provides the first **systematic survey** dedicated to MAS in SE:
 - Classifies applications across the lifecycle, synthesizes claimed benefits (divergent thinking, cross validation), and highlights **limitations and open challenges** [18].
 - The tone is more **cautious and forward looking** acknowledging that synergy and robustness are not automatic.
- In parallel, **Self Collaboration via ChatGPT**[22] matures into a widely cited reference; its first author later co authors the 2025 **survey on code generation with LLM based agents**[2], showing an emerging **research thread around multi agent code generation**.

Reasoning vs non reasoning: First explicit SE specific comparisons

In 2024 the first work clearly aligned with your (2) question appears:

- **SymPrompt / Code Aware Prompting**[8]:
 - Uses **multi stage, path guided prompts**to elicit more structured reasoning about code paths for test generation.
 - Demonstrates large improvements in both **correct tests and coverage** relative to simpler one shot prompts, across different models (CodeGen2, GPT 4) [8].
 - Conceptually, this is an early “**reasoning mode vs direct mode**”ablation in an SE task:
 - Structured, multi step reasoning helps**coverage oriented, path sensitive**tasks.
 - It sets a template later extended conceptually (e.g., SIADAFIX’s “fast vs slow thinking” [7]).

However, **most 2024 MAS papers still do not explicitly contrast CoT/deliberative vs direct modes**; they embed “thinking” as design detail rather than as an object of study.

2025: Systematic Evaluation, Failure Oriented Analyses, and Nuanced MAS Design

2025 brings a notable **shift from “can we make MAS work?” to “when, why, and at what cost?”** The key developments include broad empirical evaluations, failure taxonomies, and proposals that directly tackle known weaknesses.

Large scale empirical evaluations of software agents

- **Comprehensive Empirical Evaluation of Agent Frameworks on Code Centric SE Tasks**[1]:
 - Benchmarks **seven general purpose agent frameworks**on:
 - Software development,
 - Vulnerability detection,
 - Program repair.
 - Directly compares **multi agent vs single agent architectures**across:
 - Effectiveness, efficiency, monetary/token overhead.
 - Key findings:
 - Overall performance is **moderate**, with no consistent dominance of multi agent systems [1].
 - **Multi agent underperformance is explicitly traced to**
 - **Interaction/coordination overhead** (long trajectories, many correction attempts).
 - **Token growth ' context overflows ' information loss.**
 - **Inter agent hallucinations and error propagation** particularly malformed diffs or repeated incorrect patches.

- **Single agent pipelines** often:
 - Maintain more compact contexts,
 - Preserve historical information better,
 - Exhibit better control over hallucinations / false positives [1].
- This paper is a **milestone**: it provides one of the first *broad, systematic, empirically grounded* arguments for why multi agent setups often fail to dominate single agent ones in code tasks.
- **SWE Perf**[14]:
 - Introduces a benchmark for **performance optimization** in real world repositories.
 - Compares:
 - An **Agentless style pipeline** and
 - **OpenHands**, an agentic platform with iterative reasoning and multi step interactions, using the **same underlying model** (Claude 3.7 Sonnet) [14].
 - Shows a **substantial gap** between all current methods and human experts, and uses this to argue that **neither agentic complexity nor simple pipelines are yet sufficient**.
 - This extends the agent vs agentless discussion to **performance sensitive optimization** not just functional correctness.

Failure taxonomies and “cognitive deadlocks”

- **Empirical Study on Failures in Automated Issue Solving** [4]:
 - Analyzes 150 failed SWE Bench Verified instances for agentic and pipeline based tools.
 - Derives a **taxonomy of failures** across three phases (understanding, reproduction, repair) with 9 primary categories and 25 subcategories.
 - Finds **distinct “failure fingerprints”**:
 - Agentic (ReAct style) systems often hit **“cognitive deadlocks”**, where the agent cycles through unproductive actions without revising its high level plan [4].
 - Pipeline methods suffer more from **rigid workflows** unable to adapt to unexpected error modes.
 - Proposes a **Collaborative Expert–Executor architecture** (a supervising Expert guiding an Executor) that recovers 22.2% of previously unsolved OpenHands cases on a 108 issue subset [4].
 - This is important as **it frames multi agent benefits in terms of specific roles and oversight** not just “more agents,” and shows **where MAS still fail**.
- **AgentAsk: Multi Agent Systems Need to Ask**[5]:
 - Diagnoses **“edge level” error cascades** at inter agent handoffs as a main MAS failure mode.
 - Proposes a plug and play **AgentAsk module** that learns *when/what/whom/how* to ask clarifying questions between agents, trained from failure traces and optimized with E GRPO [5].
 - Demonstrates **consistent robustness gains on coding and reasoning benchmarks** with <5% overhead.
 - Conceptually, this:
 - Makes explicit that **inter agent communication is a first class source of errors** not just a neutral channel.
 - Reinforces that MAS require **learned coordination policies**, not just static debate or role prompts.
- **Is Multi Agent Debate (MAD) the Silver Bullet?**[6]:
 - Applies **multi agent debate** to code summarization and translation.
 - Concludes that structured debate can improve performance in some cases but highlights **open issues and limitations**, undermining the idea of MAD as universally beneficial.
 - This mirrors similar results in math/logic domains, now in SE tasks.

Domain specific MAS for SE tasks

Several 2025 papers refine MAS design for specific SE problems:

- **RefAgent** [12]: multi agent refactoring with planner / executor / tester / refiner.
 - Explicitly compares to:
 - A **single agent approach**
 - A search based refactoring tool,
 - Historical developer refactorings.

Shows improved quality and better behavior preservation via **role specialization + tool calling** but still within the broader picture where MAS are **one design point among others**, not clearly dominant.

- **Evaluating Classical Software Process Models as Coordination Mechanisms** [13]:
 - Organizes MAS by **Waterfall, V Model, Agile** coordination patterns.
 - Finds that:
 - Agile style iterative coordination yields **higher code quality but greater computational cost**.
 - Waterfall is more efficient but produces smaller, lower quality codebases [13].
 - This recasts MAS design as **software process engineering at the agent level** emphasizing **coordination structure** and its cost–quality trade offs.
- **IJRASET multi agent bug detection and refactoring** [11]:
 - Integrates static analysis, test generation, root cause analysis, and refactoring agents under a planner.
 - Reports higher fix precision and reduced review time vs single agent and non LLM baselines [11].
 - Includes ablations on **agent roles and communication cost**, underlining that **coordination depth** affects utility.
- **Preliminary Study on LLM Self Negotiation** [3]:
 - Introduces **self negotiation** (agents of the same model debating) for:
 - Equivalent mutant detection,
 - Automated vulnerability detection,
 - Automated program repair [3].
 - Finds:
 - Very large gains (114–351%) for equivalent mutant detection.
 - Mixed results for vulnerability detection and **no strong improvement for APR** (comparable to single agent).
 - This is directly relevant to your question: **debate style multi agent setups can be highly beneficial for some code understanding tasks but not for program repair**, highlighting task dependent value of additional reasoning/negotiation.
- **When Disagreements Elicit Robustness** [23]:
 - Studies MAS under deliberate **knowledge conflicts** on:
 - **Programming tasks** (HumanEval, GAIA) vs
 - **Single path reasoning tasks** (CounterFact, MQuAKE cf).
 - Shows that:
 - Heterogeneity generally boosts robustness.
 - Task critical conflicts are much more damaging on **single path reasoning** than on **multi implementation programming tasks**; MAS can often “bypass” wrong facts in programming [23].
 - This links **task structure** (single vs multiple valid paths) to MAS robustness, adding nuance to where diverse agents help or hurt.

Reasoning vs non reasoning: Fast vs slow thinking in program repair

- **SIADAFIX** [7]:
 - Proposes an **adaptive program repair framework** that combines:
 - **“Fast thinking”**: single or hard coded LLM calls for simpler issues.
 - **“Slow thinking”**: autonomous, iterative repair loops for harder cases.
 - Uses an issue description classifier to route issues to **easy/middle/hard** workflows; slow thinking is reserved for complex problems [7].
 - Motivated explicitly by **“continuous error accumulation”** in slow, autonomous agents; SIADAFIX aims to **limit slow - thinking when it does more harm than good**.
 - This is an explicit **reasoning mode vs fast mode distinction** within a single system, with adaptive selection based on problem complexity.

Together, [4,5,6,7,1,3,23] mark a **clear methodological shift**:

- From “build clever agent systems” ’
- To **systematically measuring when multi agent vs single agent vs fast/slow reasoning modes help or hurt**, and how **task structure, coordination strategy, and feedback** mediate those outcomes.

Evolution of Views on Reasoning vs Direct / Instruction Following Modes

While fewer papers isolate reasoning modes as cleanly as multi agent vs single agent, we can trace a clear conceptual evolution:

Stage 1: Implicit assumption that more reasoning is better (H2023–early 2024)

- Early multi agent and CoT infused systems (e.g., AgentCoder [21], self collaboration [22]) implicitly **equated better performance with more explicit reasoning**, longer trajectories, and richer discussions.
- Chain of thought, test driven loops, and extended dialogues **are baked into the system** without comparison to direct answer modes.

Stage 2: Evidence that structured reasoning helps specific SE tasks

- SymPrompt [8] establishes that:
 - For **coverage oriented test generation** multi stage, path aware prompting (a reasoning mode) **significantly outperforms** simple prompts.
 - This reinforces the idea that **structured reasoning is valuable when tasks involve non local, path dependent requirements**.

Stage 3: Emergence of fast vs slow thinking and adaptive reasoning

- SIADAFIX [7] leverages dual process (fast/slow) thinking explicitly:
 - Fast, direct modes are preferred for **simple, pattern like bug fixes** where extended reasoning tends to accumulate errors.
 - Slow, deliberative (agentic) modes are applied only to **complex issues** requiring multi step reasoning.
 - The design and empirical results embody a key insight: **reasoning modes are not uniformly beneficial**; their value is strongly **conditioned on problem difficulty and structure**.
- Self negotiation results[3] further sharpen this:
 - Negotiation (a form of extended meta reasoning) **boosts equivalent mutant detection**, somewhat helps or hurts **vulnerability detection** depending on model, and **adds little for APR**.
 - This aligns with the intuition that:
 - **Understanding / classification tasks** with ambiguous semantics may benefit from multiple views/negotiation.
 - **Repair tasks** grounded in tests often benefit more from **local search + execution feedback** than from extra layers of abstract reasoning.

Stage 4: Integration of reasoning with external tools and tests

Across many works (AgentCoder [21], RefAgent [12], MAGIS [9], OpenHands and Agentless [14,15,16]) a common pattern emerges:

- **Execution feedback (compilers, tests, static analysis)** increasingly dominates **internal reasoning** as the primary correctness signal.
- Explicit reasoning (CoT, planning traces, debates) becomes **auxiliary**:
 - Helpful for **high level decomposition** or interpreting error messages.
 - Less central when **strong external verifiers** exist—simple generate–test–fix loops suffice for many local bugs.

This supports a broad, emerging consensus in the literature:

- **Reasoning/thinking modes** are:
 - Most useful for **multi step planning, code understanding, or multi file coordination** without strong tests.
 - Potentially harmful or wasteful for **pattern based, locally testable tasks** where they add verbosity and risk “locking in” wrong designs.
- **Direct/instruction following modes**
 - Remain **competitive or superior** for:
 - Straightforward code generation.
 - Many small program repairs guided by precise error messages.
 - Provide better **compute efficiency**, allowing more sampling or deeper exploration within the same budget.

Collaborator Clusters and Sustained Research Threads

Several author groups appear repeatedly and shape the field's trajectory.

Lingming Zhang's group (Agentless, DEMystification, SWE Perf, DEI)

- **Agentless** [16] and its journal version [15] are authored by **Chun Xia et al.** under Lingming Zhang, with:

- Clear empirical evidence that **simple, non agentic pipelines can outperform complex agentson** SWE Bench Lite.
- Strong arguments about **cost, complexity, and reproducibility**.
- **DEI** [19] comes from a related ecosystem of SWE agents and is cited by [15,16,18].
 - It explores **committee orchestration** on top of existing agents.
- **SWE Perf**[14] (co authored by Zejun Ma et al., with Xia & Zhang) extends these themes to **performance optimization**, keeping the **Agentless vs agentic** contrast explicit.
- Their work is also **referenced and built on** by:
 - The comprehensive agent evaluation [1],
 - Failure analyses [4],
 - The MAS survey [18].

Impact: This cluster has **reframed the field's baseline**, forcing new MAS proposals to justify their complexity against **strong agentless baselines** and to report **cost/efficiency** as first class metrics.

Ge Li / Yihong Dong's thread (self collaboration, code agent survey)

- **Self Collaboration via ChatGPT**[22] is one of the **earliest and most cited** code multi agent frameworks.
- **Survey on Code Generation with LLM based Agents**[2] (2025) consolidates:
 - Single agent vs multi agent architectures.
 - Patterns like decomposition ' coding ' debugging ' verification.
 - Documented **failure modes** such as state drift and error propagation [2].
- This group thus provides:
 - Both **initial optimism** (self collaboration) and
 - A later **systematic taxonomy and synthesis** of MAS design patterns and weaknesses.

David Lo / Junda He's MAS for SE vision

- **LLM Based Multi Agent Systems for SE**[18] (TOSEM 2024) is the **first broad survey** dedicated to MAS in SE.
 - Catalogues applications, articulates claimed benefits, and sets a **research agenda** focused on agent synergy and robustness.
- This vision piece is **cited by later empirical and process oriented works**[13,18], anchoring MAS research within the SE community's concerns (trust, verification, lifecycle coverage).

Quality/failure analysis cluster

- **Xia et al.** [15,16], **Meng et al.** [17], **Liu et al.** [4], and **Yin et al.** [1] collectively form a **methodological pivot**:
 - They emphasize **benchmark rigor** (SWE Bench / SWE Bench Verified / SWE Perf)
 - Careful **failure categorization**,
 - And **comparisons across multiple tools and architectures**.
- This cluster's work **encourages more nuanced agent designs** (e.g., collaborative Expert–Executor [4]) and **discourages purely anecdotal agent success stories**.

Reasoning mode innovation cluster

- **SymPrompt** [8] and **SIADAFIX** [7] exemplify a smaller, but important line of work that:
 - Treats **reasoning vs non reasoning** as an explicit variable.
 - Designs **prompt/program structures** and **routing mechanisms** to match reasoning depth to task complexity.

Implications for the future:

- **Agentless vs MAS** will likely remain a central axis, with increasingly **compute matched evaluations** and **standardized benchmarks**.
- We should expect:
 - More **adaptive systems** that dynamically allocate agents and reasoning depth based on **learned estimates of task hardness**.
 - Deeper integration of **verification tools** and **learned coordination policies** (e.g., AgentAsk style learned questioning [5]) rather than static role/debate prompts.

Emerging Patterns and Likely Future Directions

Synthesizing across the timeline:

- **Trend 1: From enthusiasm to cost aware skepticism.**
 - **Early stage** (2023–early 2024): MAS framed as natural, powerful extensions of LLM capabilities in coding, often compared only to weak single shot baselines [20,21,22].
 - **Mid stage** (~2024): Repository scale benchmarks and surveys highlight both benefits and strong limitations [8,9,17,18,19].
 - **Recent stage** (2025): Rigorous evaluations show **no guaranteed MAS dominance**, and sometimes clear **agentless superiority** under realistic budgets [1,4,14,15,16].
- **Trend 2: Increasing importance of coordination and communication modeling.**
 - Simple role prompts and naive debates are no longer viewed as sufficient.
 - New work explicitly models:
 - **Edge level errors and question asking policies** [5].
 - **Process model inspired coordination** (Agile/Waterfall/V Model) [13].
 - **Expert–Executor supervision structures** [4].
- **Trend 3: Task specific value of multi agent reasoning.**
 - MAS / self negotiation are beneficial for:
 - **Code understanding / mutant detection** [3].
 - Some **repository scale issue resolution** tasks [9,12].
 - They show **mixed or limited benefit** for:
 - **APR** when strong tests and local patches dominate [3,7,15,16,17].
 - Tasks with strong external feedback where extra reasoning mostly adds cost.
- **Trend 4: Nuanced view of reasoning vs direct modes.**
 - Structured reasoning helps:
 - **Path sensitive test generation** [8].
 - **Complex, multi file or design level repair** [7,12].
 - Direct modes excel on:
 - **Short, pattern based code generation**
 - **Local bug fixes** guided by clear error messages or tests [7,15,16,17].
 - Adaptive fast/slow mechanisms (e.g., SIADAFIX [7]) suggest a future in which **reasoning depth becomes a controllable, task conditioned resource**
- **Trend 5: Increasing use of learned meta policies.**
 - From static flows (hard coded MAS pipelines) toward:
 - **Learned routing/classification** (problem difficulty ' fast vs slow route) [7].
 - **Learned communication policies** for clarification and information sharing [5].
 - This indicates a trajectory where **meta decision making about agents and reasoning** becomes as important as the base LLM's coding ability.

In sum, the field has moved from **unquestioned optimism about multi agent “LLM teams”** to a more **mature, evidence driven perspective**:

- Multi agent setups **sometimes help**, particularly for **complex understanding tasks and large scale refactorings** but come with **nontrivial coordination and cost overheads** and **no universal performance guarantees** [1,3,4,9,12,18].
- Carefully designed **single agent pipelines** and **adaptive reasoning modes** often match or beat MAS on **bug fixing and performance optimization**, especially under realistic compute constraints [7,14,15,16,17].
- Future progress is likely to come from:
 - **Adaptive, tool grounded orchestration** (learned policies deciding when to reason, when to debate, when to call tools),
 - Rather than purely increasing the number of agents or blindly adding more chain of thought to software engineering tasks.

Foundational Work

Which papers form the foundational references on this topic?

The below table shows the resources that are most often cited by the relevant papers on this topic. This is measured by the **reference rate**, which is the fraction of relevant papers that cite a resource. Use this table to determine the most important core papers to be familiar with if you want to deeply understand this topic. Some of these core papers may not be directly relevant to the topic, but provide important context.

Ref.	Reference Rate	Title	Cited By These Relevant Papers
[16]	0.45	Agentless: Demystifying LLM-based Software Engineering Agents	[1, 4, 6, 7, 12, 14]
[141]	0.27	Program Synthesis with Large Language Models	[5, 8, 9, 13, 16]
[79]	0.26	Evaluating Large Language Models Trained on Code	[8, 9, 10, 15, 16]
[69]	0.25	SWE-bench: Can Language Models Resolve Real-World GitHub Issues?	[7, 9, 13, 15, 16]
[96]	0.24	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	[1, 7, 15, 16]
[120]	0.21	Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate	[5, 6, 10, 18]
[28]	0.20	ChatDev: Communicative Agents for Software Development	[5, 6, 13, 18]
[47]	0.20	Large Language Models for Software Engineering: A Systematic Literature Review	[3, 7, 9]
[99]	0.20	ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate	[5, 6, 9]
[50]	0.17	OpenHands: An Open Platform for AI Software Developers as Generalist Agents	[4, 7]
[142]	0.14	A survey on large language model based autonomous agents	[6, 10, 18]
[56]	0.14	RepairAgent: An Autonomous, LLM-Based Agent for Program Repair	[9, 15, 16]
[143]	0.13	Self-Refine: Iterative Refinement with Self-Feedback	[5, 6]
[48]	0.13	AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology	[6, 13, 18]
[26]	0.13	Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling	[7]
[15]	0.11	Demystifying LLM-Based Software Engineering Agents	[4, 17]
[35]	0.11	Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories	[7]
[34]	0.11	Understanding Software Engineering Agents Through the Lens of Traceability: An Empirical Study	[1]
[54]	0.10	EXPERPAIR: Dual-Memory Enhanced LLM-based Repository-Level Program Repair	[7]
[77]	0.10	Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT	[7, 16]

Adjacent Work

Which papers cite the same foundational papers as relevant papers?

Use this table to discover related papers on adjacent topics, to gain a broader understanding of the field and help generate ideas for useful new research directions.

Ref.	Adjacency score	Title	References These Foundational Papers
[63]	0.20	Self-Abstraction from Grounded Experience for Plan-Guided Policy Refinement	[16, 19, 69, 78, 79, 85]
[35]	0.11	Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories	[16, 69, 79]
[50]	0.10	OpenHands: An Open Platform for AI Software Developers as Generalist Agents	[16, 69, 79]
[16]	0.08	Agentless: Demystifying LLM-based Software Engineering Agents	[19, 69, 78, 79, 85]

[83]	0.08	Issue2Test: Generating Reproducing Test Cases from Issue Reports	[16, 69, 96]
[90]	0.07	You Name It, I Run It: An LLM Agent to Execute Tests of Arbitrary Projects	[69, 78, 96]
[34]	0.07	Understanding Software Engineering Agents Through the Lens of Traceability: An Empirical Study	[16, 69]
[15]	0.05	Demystifying LLM-Based Software Engineering Agents	[19, 69, 79]
[56]	0.04	RepairAgent: An Autonomous, LLM-Based Agent for Program Repair	[79, 96]
[21]	0.04	AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation	[79, 111, 143]
[28]	0.04	ChatDev: Communicative Agents for Software Development	[79, 99, 111]
[151]	0.04	Automated Cloud Infrastructure-as-Code Reconciliation with AI Agents	[16, 96]
[53]	0.04	InfCode: Adversarial Iterative Refinement of Tests and Patches for Reliable Software Issue Resolution	[16, 96]
[23]	0.04	When Disagreements Elicit Robustness: Investigating Self-Repair Capabilities under LLM Multi-Agent Disagreements	[28, 79]
[152]	0.04	TREAT: A Code LLMs Trustworthiness / Reliability Evaluation and Testing Framework	[16, 96]
[153]	0.03	InfCode-C++: Intent-Guided Semantic Retrieval and AST-Structured Search for C++ Issue Resolution	[16, 96]
[154]	0.03	SkyRL-Agent: Efficient RL Training for Multi-turn LLM Agent	[16, 96]
[155]	0.03	Scalable Supervising Software Agents with Patch Reasoner	[16, 96]
[156]	0.03	TheMCPCompany: Creating General-purpose Agents with Task-specific Tools	[16, 96]
[157]	0.03	BugPilot: Complex Bug Generation for Efficient Learning of SWE Skills	[16, 96]

References

- [1] [A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks](#)
Zhuowen Yin, ..., and Lijun Zhang. ArXiv, 2025. 0 citations.
100% Topic Match
Presents a large empirical comparison of seven multi-agent frameworks vs single-agent baselines on code SE tasks. Evaluates effectiveness, efficiency, and token/monetary overhead across software development, vulnerability detection, and program repair benchmarks. Finds only moderate agent performance; cites coordination/token overhead, inter-agent hallucination/error propagation, malformed patches, and context-size information loss as causes why multi-agent setups sometimes underperform single-agent approaches.
- [2] [A Survey on Code Generation with LLM-based Agents](#)
Yihong Dong, ..., and Ge Li. ArXiv, 2025. 19 citations.
100% Topic Match
Surveys LLM-based code-generation agents, categorizing single- vs multi-agent architectures. Summarizes design patterns, concrete multi-agent systems, reported SE gains (e.g., SecurityEval ~13% reduction), and documented failure modes. Relevant as a broad syntheses: it reports empirical claims but mostly aggregates others' results (check original system papers for controlled single-agent vs multi-agent comparisons and budget-controlled evaluations).
- [3] [A Preliminary Study on Large Language Models Self-Negotiation in Software Engineering](#)
Chunrun Tao, ..., and Yasutaka Kamei. 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2025. 0 citations.
100% Topic Match
Evaluates multi-agent "LLM self-negotiation" vs single LLMs on SE tasks. Compares collaborative/debate-style multiple LLMs to single-agent baselines across EMD, AVD, and APR with empirical metrics. Findings: large gains for Equivalent Mutant Detection (114–351% F1 gain; one model slight drop), comparable performance for Program Repair, and mixed/partial results for Vulnerability Detection — paper reports task-dependent effectiveness and thus is directly relevant; check whether single-agent baselines match compute/budget and whether agents are distinct LLM instances versus multi-sampling (paper likely discusses variability but details on budget/control are needed).
- [4] [An Empirical Study on Failures in Automated Issue Solving](#)
Simiao Liu, ..., and Li Zhang. ArXiv, 2025. 2 citations.
99% Topic Match
Analyzes failure modes in automated issue solving comparing pipeline, agentic, and multi-agent tools. Evaluates three SOTA tools on SWE-Bench-Verified, manually labels 150 failed instances into a 3-phase/9-category/25-subcategory taxonomy, and measures failure distributions. Finds agentic/multi-agent systems show failure fingerprints (flawed reasoning, cognitive deadlocks, coordination/rigidity); proposes Expert–Executor and reports partial recovery (24/108) and comparable performance to strong single-agent baselines.
- [5] [AgentAsk: Multi-Agent Systems Need to Ask](#)
Bohan Li, ..., and Yang Wang. ArXiv, 2025. 0 citations.
98% Topic Match
Identifies edge-level error cascades causing multi-agent underperformance. Diagnoses inter-agent handoff mistakes and proposes AgentAsk, a clarification module inserting targeted questions to stop propagation. Empirical gains reported on coding benchmarks with <5% overhead, but no clear matched single-agent baseline comparison nor reasoning-vs-non-reasoning mode analysis.
- [6] [Is Multi-Agent Debate \(MAD\) the Silver Bullet? An Empirical Analysis of MAD in Code Summarization and Translation](#)
Jina Chun, ..., and Iftekhar Ahmed. ArXiv, 2025. 5 citations.
98% Topic Match
Analyzes MAD applied to SE tasks (code summarization, code translation). Adapts NLP multi-agent debate setups with role-specific LLM agents, inspects interactions, and reports empirical results plus two proposed improvements. Relevant: focuses SE code tasks, compares multi-agent debate performance qualitatively/quantitatively, but abstract doesn't state direct single-agent baselines under matched compute or explicit comparisons of reasoning-mode vs non-reasoning-mode.
- [7] [SIADAFIX: issue description response for adaptive program repair](#)
Xin Cao and Nan Yu. ArXiv, 2025. 0 citations.
97% Topic Match
Proposes an adaptive program-repair method combining fast and slow thinking (SIADAFIX). Uses a fast workflow classifier/issue-description optimizer to route problems to a slow-thinking iterative bug-fix agent, selecting easy/middle/hard modes and applying test-time scaling. Relevant: focuses program-repair (SE task) and contrasts single fast calls vs autonomous multi-round agents; lacks explicit comparison to multi-agent (two+ interacting LLMs) or controlled reasoning-vs-non-reasoning ablations under fixed compute.
- [8] [Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM](#)
Gabriel Ryan, ..., and Baishakhi Ray. Proceedings of the ACM on Software Engineering, 2024. 113 citations.
93% Topic Match
Demonstrates a reasoning-style, code-aware prompting method (SymPrompt) improves LLM test generation. Implements multi-stage, path-aligned prompts using TreeSitter to expose types/dependencies, eliciting multi-step reasoning from a single pretrained LLM and measuring coverage gains. Relevant: empirically compares reasoning-style vs standard prompting on Python test-generation (large coverage and correctness gains); does NOT evaluate multi-agent setups or compare multi-agent vs single-agent under fixed budgets.
- [9] [MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution](#)
Wei Tao, ..., and Yu-Xi Cheng. ArXiv, 2024. 91 citations.
92% Topic Match
Proposes a tailored LLM-based multi-agent framework (MAGIS) for repository-level GitHub issue resolution. Implements four role-specific agents (Manager, Repository Custodian, Developer, QA) and evaluates on SWE-bench, comparing to single-instance LLMs (GPT-3.5/4, Claude-2). Reports large empirical gains (13.94% resolved; ~8x vs direct GPT-4), analyzes failure factors (context-length limits, repo-scale challenges); compare-to-single-agent baseline present, but paper focuses on solution design and empirical improvements rather than isolating when multi-agent may underperform or detailed reasoning-vs-non-reasoning mode ablations.
- [10] [CoopetitiveV: Leveraging LLM-powered Coopetitive Multi-Agent Prompting for High-quality Verilog Generation](#)
Zhendong Mi, ..., and Zhaozhao Xu Shaoyi Huang. ArXiv, 2024. 4 citations.
86% Topic Match
Proposes a coopetitive multi-agent LLM framework for Verilog code generation. Implements distinct agent roles (generator, strategist, challenger, corrector) mixing cooperation and competition to reduce degeneration and error propagation, reporting high pass@10 on Verilog benchmarks.

Relevant but limited: focuses on Verilog; paper identifies multi-agent failure modes and mitigation, yet abstract/excerpt does not report compute-matched single-agent baselines nor explicit comparisons of reasoning (CoT) versus non-reasoning modes.

[11] [Evaluating Multi-Agent AI Systems for Automated Bug Detection and Code Refactoring](#)

Tanveer Aamina, ..., and Syeda Huda. International Journal for Research in Applied Science and Engineering Technology, 2025. 0 citations.

86% Topic Match

Demonstrates a cooperative multi-agent pipeline for bug detection and refactoring.

Implements specialized agents (static-analysis, test-gen, root-cause, refactoring) plus a planner, integrating LLM reasoning with program analysis, compared to single-agent and non-LLM baselines on Python/Java projects.

Reports ablations on agent roles, verification depth, communication cost, and failure modes (spec ambiguity, over-refactoring, flaky tests); useful because it includes direct multi-agent vs single-agent comparisons and analysis of when orchestration helps or fails.

[12] [RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring](#)

Khoulood Oueslati, ..., and Foutse Khomh. ArXiv, 2025. 0 citations.

77% Topic Match

Proposes a multi-agent LLM framework (RefAgent) for end-to-end automated software refactoring.

Implements specialized agents (planner, executor, tester, refiner) with tool calls, self-reflection, and iterative workflows; evaluated on eight open-source Java projects. Directly compares multi-agent against a single-agent baseline and ablates agent contributions, but does not compare reasoning (CoT) versus non-reasoning modes or control for compute/selection budget in reported excerpts.

[13] [Evaluating Classical Software Process Models as Coordination Mechanisms for LLM-Based Software Generation](#)

Duc Ha Minh, ..., and Anh Nguyen-Duc. ArXiv, 2025. 0 citations.

68% Topic Match

Evaluates multi-agent LLM-based software teams organized by classical software process models.

Runs 11 projects across Waterfall, V-Model, and Agile with role-based agents and four GPT variants (132 runs), measuring size, cost, and quality.

Relevant because it empirically analyzes when multi-agent coordination patterns help or hurt code quality/cost, but it lacks a direct single-agent baseline and provides no explicit comparison of reasoning (CoT) versus non-reasoning modes.

[14] [SWE-Perf: Can Language Models Optimize Code Performance on Real-World Repositories?](#)

Xinyi He, ..., and Zejun Ma. ArXiv, 2025. 4 citations.

58% Topic Match

Introduces and evaluates SWE-Perf, a benchmark for LLM-based repository-level code performance optimization.

Builds 140 real-world instances from performance-improving GitHub PRs and compares file-level (Agentless) and agent-based (OpenHands) LLM pipelines using Claude-3.7-sonnet and other models.

Relevant: directly compares an agent-based multi-step interaction platform (OpenHands) to an agentless/single-pass pipeline; includes empirical gaps vs expert patches but may not isolate multi-agent vs single-agent under equal compute or analyze reasoning-mode vs non-reasoning-mode differences.

[15] [Demystifying LLM-Based Software Engineering Agents](#)

Chun Xia, ..., and Lingming Zhang. Proceedings of the ACM on Software Engineering, 2025. 40 citations.

56% Topic Match

Demonstrates that a simple "agentless" single-LLM pipeline can outperform complex autonomous multi-agent SE agents.

Compares Agentless (localize/repair/validate) against open-source agent systems on SWE-bench Lite, reporting higher fix rate (32%, 96 fixes) and lower cost (\$0.70).

Relevant: direct empirical multi-agent vs single-agent comparison on code/program-repair tasks, includes cost/compute and pragmatic evaluation — good fit for (1); does not explicitly state CoT vs non-CoT mode comparisons, so (2) likely not addressed.

[16] [Agentless: Demystifying LLM-based Software Engineering Agents](#)

Chun Xia, ..., and Lingming Zhang. ArXiv, 2024. 207 citations.

47% Topic Match

Demonstrates a simple single-agent "agentless" pipeline outperforming complex autonomous agents on SE tasks.

Builds a three-phase localization/repair/patch-validation pipeline (no multi-agent orchestration or agent decision loops) and evaluates on SWE-bench Lite, achieving 32.0% fixes at low cost.

Relevant: directly compares agentless single-pipeline to agent-based systems on program-repair/code tasks, includes error classification and cost/efficiency analysis — but does not test multi-agent debate variants under equal compute budgets.

[17] [An Empirical Study on LLM-based Agents for Automated Bug Fixing](#)

Xiangxin Meng, ..., and Chao Peng. ArXiv, 2024. 13 citations.

33% Topic Match

Evaluates LLM-based agent vs non-agent systems for automated bug fixing.

Compares seven top repair systems on SWE-bench (agentic and agentless), measuring pass rates, overlap of solvable instances, fault-localization accuracy, and reproduction impact.

Finds agents do not uniformly outperform non-agents; attributes failures to LLM reasoning limits, agent-flow design issues (localization, multi-location fixes, verification, output randomness), and recommends better grounding/tooling and agent coordination.

[18] [LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead](#)

Junda He, ..., and David Lo. ACM Transactions on Software Engineering and Methodology, 2024. 79 citations.

29% Topic Match

Conducts a systematic review and visioning of LLM-based multi-agent systems (LMAS) for software engineering.

Surveys recent primary studies, presents two case studies demonstrating LMA capabilities and limits, and proposes a research agenda.

Mostly conceptual/systematic-review: discusses claimed benefits (debate, specialization, validation) and limitations, but lacks empirical comparisons under fixed budgets or direct single-agent vs multi-agent controlled experiments on SE tasks.

[19] [Diversity Empowers Intelligence: Integrating Expertise of Software Engineering Agents](#)

Kexun Zhang, ..., and Caiming Xiong. ArXiv, 2024. 32 citations.

17% Topic Match

Proposes DEI, a meta-module that orchestrates diverse SWE LLM agents into committees.

Shows empirically that DEI-guided agent committees improve patch-resolve rates on SWE-Bench Lite (27.3%→34.3%, best group 55%).

Lacks controlled comparisons against tuned single-agent baselines under equal compute/latency, no analysis of multi-agent failure cases, and no reasoning-vs-non-reasoning mode experiments.

[20] [Experiential Co-Learning of Software-Developing Agents](#)

Cheng Qian, ..., and Maosong Sun. ArXiv, 2023. 63 citations.

13% Topic Match

Proposes a multi-agent "Experiential Co-Learning" framework for software-development agents.

Implements instructor+assistant agents that store/reuse past interaction trajectories and benchmarks against GPT-Engineer, MetaGPT, ChatDev on NLDD.

Empirically shows Co-Learning improves completeness/executability and is more time-efficient than other multi-agent systems, outperforms single-agent GPT-Engineer (attributed to decomposition), but lacks controlled CoT vs non reasoning comparisons, fixed-budget ablations, or detailed failure-mode analysis explaining when multi-agent may underperform.

[21] [AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation](#)

Dong Huang, ..., and Heming Cui. ArXiv, 2023. 245 citations.

9% Topic Match

- Proposes a multi-agent code-generation system (Programmer, Test Designer, Test Executor).
Implements iterative generation + auto-test creation and execution, feeding error traces back for repairs; evaluated on HumanEval and MBPP with large pass@1 gains.
Lacks clear single-agent compute/latency-parity baselines and detailed failure analyses; notes tester can misjudge correctness but gives limited explanation when multi-agent might underperform.
- [22] [Self-Collaboration Code Generation via ChatGPT](#)
Yihong Dong, ..., and Ge Li. ACM Transactions on Software Engineering and Methodology, 2023. 374 citations.
6% Topic Match
Proposes a self-collaboration multi-agent LLM framework for code generation (analyst, coder, tester).
Implements role-based ChatGPT agents that interact via prescribed protocols and software-development workflows; reports 29.9–47.1% relative Pass@1 gains on benchmarks.
Relevant caveats: uses single LLM via role prompts (not distinct model instances), compares to a base agent but paper may lack fixed-budget single-agent-with-sampling or ablations isolating role vs extra compute; useful for multi-agent design but may not fully analyze failure modes where multi-agent underperforms.
- [23] [When Disagreements Elicit Robustness: Investigating Self-Repair Capabilities under LLM Multi-Agent Disagreements](#)
Tianjie Ju, ..., and Gongshen Liu. ArXiv, 2025. 1 citations.
5% Topic Match
Investigates robustness of multi-agent LLM systems under injected disagreements.
Runs experiments on collaborative reasoning (CounterFact, MQuAKE-cf) and programming (HumanEval, GAIA), injecting heterogeneous vs task-critical conflicts and measuring success.
Relevant because it empirically shows when MAS helps (general heterogeneity) versus hurts (task-critical edits on single-path reasoning), but it lacks direct matched-compute single-agent baselines and does not compare reasoning-mode versus non-reasoning/instruction-following modes.
- [24] [SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement](#)
Antonis Antoniadis, ..., and W. Wang. ArXiv, 2024. 48 citations.
4% Topic Match
Abstract: Software engineers operating in complex and dynamic environments must continuously adapt to evolving requirements, learn iteratively from experience, and reconsider their approaches based on new insights. However, current large language model (LLM)-based software agents often follow linear, sequential processes that prevent backtracking and exploration of alternative solutions, limiting their ability to rethink their strategies when initial approaches prove ineffective. To address these challenges, we propose SWE-Search, a multi-agent framework that integrates Monte Carlo Tree Search (MCTS) with a self-improvement mechanism to enhance software agents' performance on repository-level software tasks. SWE-Search extends traditional MCTS by incorporating a hybrid value function that...
- [25] [Debate, Verify, and Debug: A Multi-Agent Planning Framework for Reliable Code Generation](#)
Kaicun Lin, ..., and Huaqiang Yuan. 2025 IEEE 8th International Conference on Computer and Communication Engineering Technology (CCET), 2025. 0 citations.
4% Topic Match
Abstract: Recent advancements in Large Language Models (LLMs) have demonstrated remarkable capabilities in generating problem-solving code directly from natural language descriptions of programming tasks. This ability highlights the potential of LLMs in automated software development. However, in real-world scenarios, user-provided problem descriptions are often incomplete or imprecise, lacking critical details required for accurate code generation. To address this limitation, we leverage the role-playing capability of LLMs and propose a multiagent framework designed to enhance the clarity, accuracy, and robustness of code generation. Our framework employs three specialized agents that collaboratively analyze the programming problem from different perspectives: requirement clarification, solution feasibility assessment,...
- [26] [Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling](#)
Pengfei Gao, ..., and Xia Liu. ArXiv, 2025. 21 citations.
3% Topic Match
Abstract: Software issue resolution is a critical challenge in software engineering and has garnered increasing attention in recent years. With the rapid advancement of large language models (LLMs), substantial progress has been made in addressing real-world software engineering tasks. Recent studies have introduced ensemble reasoning techniques to enhance the performance of LLM-based issue resolution. However, existing prompting-based methods still face limitations in effectively exploring large ensemble spaces and lack the capacity for repository-level understanding, both of which constrain their overall effectiveness. In this paper, we propose Trae Agent, the first agent-based ensemble reasoning approach for repository-level issue resolution. Trae Agent formulates...
- [27] [Context Engineering for Multi-Agent LLM Code Assistants Using Elicit, NotebookLM, ChatGPT, and Claude Code](#)
Muhammad Haseeb Bhatti. ArXiv, 2025. 1 citations.
3% Topic Match
Abstract: Large Language Models (LLMs) have shown promise in automating code generation and software engineering tasks, yet they often struggle with complex, multi-file projects due to context limitations and knowledge gaps. We propose a novel context engineering workflow that combines multiple AI components: an Intent Translator (GPT-5) for clarifying user requirements, an Elicit-powered semantic literature retrieval for injecting domain knowledge, NotebookLM-based document synthesis for contextual understanding, and a Claude Code multi-agent system for code generation and validation. Our integrated approach leverages intent clarification, retrieval-augmented generation, and specialized sub-agents orchestrated via Claude's agent framework. We demonstrate that this method significantly improves the...
- [28] [ChatDev: Communicative Agents for Software Development](#)
Cheng Qian, ..., and Maosong Sun. Unknown journal, 2023. 418 citations.
3% Topic Match
Abstract: Software development is a complex task that necessitates cooperation among multiple members with diverse skills. Numerous studies used deep learning to improve specific phases in a waterfall model, such as design, coding, and testing. However, the deep learning model in each phase requires unique designs, leading to technical inconsistencies across various phases, which results in a fragmented and ineffective development process. In this paper, we introduce ChatDev, a chat-powered software development framework in which specialized agents driven by large language models (LLMs) are guided in what to communicate (via chat chain) and how to communicate (via communicative dehallucination). These agents...
- [29] [SEW: Self-Evolving Agentic Workflows for Automated Code Generation](#)
Siwei Liu, ..., and Zaiqiao Meng. ArXiv, 2025. 5 citations.
3% Topic Match
Abstract: Large Language Models (LLMs) have demonstrated effectiveness in code generation tasks. To enable LLMs to address more complex coding challenges, existing research has focused on crafting multi-agent systems with agentic workflows, where complex coding tasks are decomposed into sub-tasks, assigned to specialized agents. Despite their effectiveness, current approaches heavily rely on hand-crafted agentic workflows, with both agent topologies and prompts manually designed, which limits their ability to automatically adapt to different types of coding problems. To address these limitations and enable automated workflow design, we propose $\text{self-evolving workflow}$ (SEW), a novel self-evolving framework that automatically generates and optimises multi-agent workflows....
- [30] [UniDebugger: Hierarchical Multi-Agent Framework for Unified Software Debugging](#)
Cheryl Lee, ..., and Michael R. Lyu. ArXiv, 2024. 60 citations.
2% Topic Match
Abstract: Software debugging is a time-consuming endeavor involving a series of steps, such as fault localization and patch generation, each requiring thorough analysis and a deep understanding of the underlying logic. While large language models (LLMs) demonstrate promising potential in coding tasks, their performance in debugging remains limited. Current LLM-based methods often focus on isolated steps and struggle with complex bugs. In this paper, we propose the first end-to-end framework, FixAgent, for unified debugging through multi-agent synergy. It mimics the entire cognitive processes of developers, with each agent specialized as a particular component of this process rather than mirroring the actions...
- [31] [CodeCoR: An LLM-Based Self-Reflective Multi-Agent Framework for Code Generation](#)

Ruwei Pan, ..., and Chao Liu. ArXiv, 2025. 23 citations.

2% Topic Match

Abstract: Code generation aims to produce code that fulfills requirements written in natural languages automatically. Large language Models (LLMs) like ChatGPT have demonstrated promising effectiveness in this area. Nonetheless, these LLMs often fail to ensure the syntactic and semantic correctness of the generated code. Recently, researchers proposed multi-agent frameworks that guide LLMs with different prompts to analyze programming tasks, generate code, perform testing in a sequential workflow. However, the performance of the workflow is not robust as the code generation depends on the performance of each agent. To address this challenge, we propose CodeCoR, a self-reflective multi-agent framework that evaluates the...

[32] [Between Promise and Pain: The Reality of Automating Failure Analysis in Microservices with LLMs](#)

Alessandro Cornacchia, ..., and Marco Canini. Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems, 2025. 1 citations.

2% Topic Match

Abstract: Large Language Models (LLMs) are increasingly explored as general-purpose assistants for infrastructure operations, helping automate tasks like querying data, analyzing logs, and suggesting fixes. In this paper, we consider the more general and ambitious problem of fully automating root cause analysis (RCA) in microservice systems, where LLMs must collect information, reason about it, and interact with the environment to detect, localize and resolve issues. Anecdotal evidence offers useful insights and partial solutions, but the broader challenge remains unresolved. We systematically evaluate multiple LLM agent architectures across a range of incident scenarios. We study how different tool-augmented agents perform, and shed...

[33] [MetaReflection: Learning Instructions for Language Agents using Past Reflections](#)

Priyanshu Gupta, ..., and Gustavo Soares. ArXiv, 2024. 14 citations.

2% Topic Match

Abstract: The popularity of Large Language Models (LLMs) have unleashed a new age of Language Agents for solving a diverse range of tasks. While contemporary frontier LLMs are capable enough to power reasonably good Language agents, the closed-API model makes it hard to improve in cases they perform sub-optimally. To address this, recent works have explored techniques to improve their performance using self reflection and prompt optimization techniques. While techniques like self reflection work well in an online setup, contemporary prompt optimization techniques are designed to work on simpler tasks. To address this, we introduce METAREFLECTION, a novel offline reinforcement learning...

[34] [Understanding Software Engineering Agents Through the Lens of Traceability: An Empirical Study](#)

Ira Ceka, ..., and Baishakhi Ray. ArXiv, 2025. 3 citations.

2% Topic Match

Abstract: With the advent of large language models (LLMs), software engineering agents (SWE agents) have emerged as a powerful paradigm for automating a range of software tasks -- from code generation and repair to test case synthesis. These agents operate autonomously by interpreting user input and responding to environmental feedback. While various agent architectures have demonstrated strong empirical performance, the internal decision-making workflows that drive their behavior remain poorly understood. Deeper insight into these workflows hold promise for improving both agent reliability and efficiency. In this work, we present the first systematic study of SWE agent behavior through the lens of...

[35] [Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories](#)

Islem Bouzenia and Michael Pradel. ArXiv, 2025. 5 citations.

1% Topic Match

Abstract: Large Language Model (LLM)-based agents are increasingly employed to automate complex software engineering tasks, such as program repair and issue resolution. These agents operate by autonomously generating natural language thoughts, invoking external tools, and iteratively refining their solutions. Despite their widespread adoption, the internal decision-making processes of these agents remain largely unexplored, limiting our understanding of their operational dynamics and failure modes. In this paper, we present a large-scale empirical study of the thought-action-result trajectories of three state-of-the-art LLM-based agents: RepairAgent, AutoCodeRover, and OpenHands. We unify their interaction logs into a common format, capturing 120 trajectories and 2,822 LLM interactions...

[36] [Evaluating Agent-Based Program Repair at Google](#)

Pat Rondon, ..., and Satish Chandra. 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2025. 18 citations.

1% Topic Match

Abstract: Agent-based program repair offers to automatically resolve complex bugs end-to-end by combining the planning, tool use, and code generation abilities of modern LLMs. Recent work has explored the use of agent-based repair approaches on the popular open-source SWE-Bench [1], a collection of bugs from highly-rated GitHub Python projects. In addition, various agentic approaches such as SWE-Agent [2] have been proposed to solve bugs in this benchmark. This paper explores the viability of using an agentic approach to address bugs in an enterprise context. To investigate this, we curate an evaluation set of 178 bugs drawn from Google's issue tracking system....

[37] [HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale](#)

H. N. Phan, ..., and Nghi D. Q. Bui. ArXiv, 2024. 25 citations.

1% Topic Match

Abstract: Large Language Models (LLMs) have revolutionized software engineering (SE), showcasing remarkable proficiency in various coding tasks. Despite recent advancements that have enabled the creation of autonomous software agents utilizing LLMs for end-to-end development tasks, these systems are typically designed for specific SE functions. We introduce HyperAgent, an innovative generalist multi-agent system designed to tackle a wide range of SE tasks across different programming languages by mimicking the workflows of human developers. HyperAgent features four specialized agents-Planner, Navigator, Code Editor, and Executor-capable of handling the entire lifecycle of SE tasks, from initial planning to final verification. HyperAgent sets new benchmarks in...

[38] [Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios](#)

Zhi Chen and Lingxiao Jiang. 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2024. 5 citations.

1% Topic Match

Abstract: In recent years, AI-based software engineering has progressed from pre-trained models to advanced agentic workflows, with Software Development Agents representing the next major leap. These agents, capable of reasoning, planning, and interacting with external environments, offer promising solutions to complex software engineering tasks. However, while much research has evaluated code generated by large language models (LLMs), comprehensive studies on agent-generated patches, particularly in real-world settings, are lacking. This study addresses that gap by evaluating 4,892 patches from 10 top-ranked agents on 500 real-world GitHub issues from SWE-Bench Verified, focusing on their impact on code quality. Our analysis shows no single...

[39] [CONSENSAGENT: Towards Efficient and Effective Consensus in Multi-Agent LLM Interactions Through Sycophancy Mitigation](#)

Priya Pitre, ..., and Xuan Wang. Unknown journal, 2025. 6 citations.

1% Topic Match

Abstract: Multi-agent large language model (LLM) systems have shown remarkable performance in tasks such as reasoning, planning, and decision-making. However, their applicability is limited by challenges such as high computational costs and robustness issues. In this work, we identify and systematically evaluate a critical yet overlooked challenge: sycophancy, where agents reinforce each other's responses instead of critically engaging with the debate. This behavior inflates computational costs by requiring additional debate rounds to reach consensus, limiting the efficiency of multi-agent LLM systems. Through experiments on six benchmark reasoning datasets across three models, we analyze the impact of sycophancy and its role in...

[40] [SWE-Effi: Re-Evaluating Software AI Agent System Effectiveness Under Resource Constraints](#)

Zhiyu Fan, ..., and Ahmed E. Hassan. ArXiv, 2025. 0 citations.

1% Topic Match

Abstract: The advancement of large language models (LLMs) and code agents has demonstrated significant potential to assist software engineering (SWE) tasks, such as autonomous issue resolution and feature addition. Existing AI for software engineering leaderboards (e.g., SWE-bench) focus solely on solution accuracy, ignoring the crucial factor of effectiveness in a resource-constrained world. This is a universal problem that also exists beyond software engineering tasks: any AI system should be more than correct - it must also be cost-effective. To address this gap, we introduce SWE-Effi, a set of new metrics to re-evaluate AI systems in terms of holistic effectiveness scores. We...

[41] [ALMAS: an Autonomous LLM-based Multi-Agent Software Engineering Framework](#)

Vali Tawosi, ..., and Xiaomo Liu. ArXiv, 2025. 2 citations.

1% Topic Match

Abstract: Multi-agent Large Language Model (LLM) systems have been leading the way in applied LLM research across a number of fields. One notable area is software development, where researchers have advanced the automation of code implementation, code testing, code maintenance, inter alia, using LLM agents. However, software development is a multifaceted environment that extends beyond just code. As such, a successful LLM system must factor in multiple stages of the software development life-cycle (SDLC). In this paper, we propose a vision for ALMAS, an Autonomous LLM-based Multi-Agent Software Engineering framework, which follows the above SDLC philosophy such that it may work...

[42] [AutoCodeRover: Autonomous Program Improvement](#)

Yuntong Zhang, ..., and Abhik Roychoudhury. Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024. 142 citations.

1% Topic Match

Abstract: Researchers have made significant progress in automating the software development process in the past decades. Automated techniques for issue summarization, bug reproduction, fault localization, and program repair have been built to ease the workload of developers. Recent progress in Large Language Models (LLMs) has significantly impacted the development process, where developers can use LLM-based programming assistants to achieve automated coding. Nevertheless, software engineering involves the process of program improvement apart from coding, specifically to enable software maintenance (e.g. program repair to fix bugs) and software evolution (e.g. feature additions). In this paper, we propose an automated approach for solving Github...

[43] [SCGen-Agent: Automated Generation of Smart Contract Based on Multi-agent Collaboration](#)

Yubo Ji, ..., and Yongle Chen. 2025 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC), 2025. 0 citations.

1% Topic Match

Abstract: To address the issues of inefficiency and high debugging costs in smart contract development, this paper proposes a multi-agent collaborative solution based on large language models (LLMs). In this approach, agents collaborate to accomplish tasks such as smart contract generation, auditing, and repair. First, an efficient fine-tuning method is employed to optimize the code generation model. Second, retrieval-augmented generation technology is utilized to provide vulnerability context knowledge for code repair. Finally, a feedback-driven multi-agent collaboration framework is designed to enhance code generation quality through agent-to-agent feedback. During the code repair phase, semantic prompts derived from control flow graphs guide the...

[44] [Meta-RAG on Large Codebases Using Code Summarization](#)

Vali Tawosi, ..., and Manuela Veloso. ArXiv, 2025. 2 citations.

1% Topic Match

Abstract: Large Language Model (LLM) systems have been at the forefront of applied Artificial Intelligence (AI) research in a multitude of domains. One such domain is software development, where researchers have pushed the automation of a number of code tasks through LLM agents. Software development is a complex ecosystem, that stretches far beyond code implementation and well into the realm of code maintenance. In this paper, we propose a multi-agent system to localize bugs in large pre-existing codebases using information retrieval and LLMs. Our system introduces a novel Retrieval Augmented Generation (RAG) approach, Meta-RAG, where we utilize summaries to condense codebases...

[45] [Interactive Debugging and Steering of Multi-Agent AI Systems](#)

Will Epperson, ..., and Saleema Amershi. Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, 2025. 28 citations.

0% Topic Match

Abstract: Fully autonomous teams of LLM-powered AI agents are emerging that collaborate to perform complex tasks for users. What challenges do developers face when trying to build and debug these AI agent teams? In formative interviews with five AI agent developers, we identify core challenges: difficulty reviewing long agent conversations to localize errors, lack of support in current tools for interactive debugging, and the need for tool support to iterate on agent configuration. Based on these needs, we developed an interactive multi-agent debugging tool, AGDebugger, with a UI for browsing and sending messages, the ability to edit and reset prior agent...

[46] [The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management](#)

Tobias Lindenbauer, ..., and Yaroslav Zharov. ArXiv, 2025. 1 citations.

0% Topic Match

Abstract: Large Language Model (LLM)-based agents solve complex tasks through iterative reasoning, exploration, and tool-use, a process that can result in long, expensive context histories. While state-of-the-art Software Engineering (SE) agents like OpenHands or Cursor use LLM-based summarization to tackle this issue, it is unclear whether the increased complexity offers tangible performance benefits compared to simply omitting older observations. We present a systematic comparison of these approaches within SWE-agent on SWE-bench Verified across five diverse model configurations. Moreover, we show initial evidence of our findings generalizing to the OpenHands agent scaffold. We find that a simple environment observation masking strategy halves...

[47] [Large Language Models for Software Engineering: A Systematic Literature Review](#)

Xinying Hou, ..., and Haoyu Wang. ACM Transactions on Software Engineering and Methodology, 2023. 668 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have significantly impacted numerous domains, including Software Engineering (SE). Many recent publications have explored LLMs applied to various SE tasks. Nevertheless, a comprehensive understanding of the application, effects, and possible limitations of LLMs on SE is still in its early stages. To bridge this gap, we conducted a Systematic Literature Review (SLR) on LLM4SE, with a particular focus on understanding how LLMs can be exploited to optimize processes and outcomes. We selected and analyzed 395 research articles from January 2017 to January 2024 to answer four key Research Questions (RQs). In RQ1, we categorize different LLMs...

[48] [AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology](#)

Minh Huynh Nguyen, ..., and Nghi D. Q. Bui. 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), 2024. 31 citations.

0% Topic Match

Abstract: Software agents are emerging as powerful tools for tackling complex software engineering tasks. However, existing approaches often oversimplify development workflows, assuming basic models that lack the nuances of real-world software processes. Moreover, they frequently include entire codebases in their instructions, leading to inefficiencies when working with large-scale projects. To overcome these challenges, we introduce AgileCoder, a multi-agent system that incorporates Agile Methodology (AM) principles, assigning specialized agents to roles such as Product Manager, Developer, and Tester for collaborative, iterative development. AgileCoder structures work into sprints, enabling incremental progress based on user input. A standout feature, the Dynamic Code Graph Generator...

[49] [From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future](#)

Haolin Jin, ..., and Huaming Chen. ArXiv, 2024. 72 citations.

0% Topic Match

Abstract: With the rise of large language models (LLMs), researchers are increasingly exploring their applications in various vertical domains, such as software engineering. LLMs have achieved remarkable success in areas including code generation and vulnerability detection. However, they also exhibit numerous limitations and shortcomings. LLM-based agents, a novel technology with the potential for Artificial General Intelligence (AGI), combine LLMs as the core for decision-making and action-taking, addressing some of the inherent limitations of LLMs such as lack of autonomy and self-improvement. Despite numerous studies and surveys exploring the possibility of using LLMs in software engineering, it lacks a clear...

[50] [OpenHands: An Open Platform for AI Software Developers as Generalist Agents](#)

Xingyao Wang, ..., and Graham Neubig. International Conference on Learning Representations, 2024. 275 citations.

0% Topic Match

Abstract: Software is one of the most powerful tools that we humans have at our disposal; it allows a skilled programmer to interact with the world in complex and profound ways. At the same time, thanks to improvements in large language models (LLMs), there has also been a rapid development in AI agents that interact with and affect change in their surrounding environments. In this paper, we introduce OpenHands (f.k.a. OpenDevin), a platform for the development of powerful and flexible AI agents that interact with the world in similar ways to those of a human developer: by writing code, interacting with...

[51] [Refactoring Python Code with LLM-Based Multi-Agent Systems: An Empirical Study in ML Software Projects](#)

Alexander Puma Pucho, ..., and Bruno Cafeo. Unknown journal, 2025. 0 citations.

0% Topic Match

Abstract: Refactoring is essential for improving software maintainability, yet it often remains a validation-intensive and developer-guided task—particularly in Python projects shaped by fast-paced experimentation and iterative workflows, as is common in the machine learning (ML) domain. Recent advances in large language models (LLMs) have introduced new possibilities for automating refactoring, but many existing approaches rely on single-model prompting and lack structured coordination or task specialization. This study presents an empirical evaluation of a modular LLM-based multi-agent system (LLM-MAS), orchestrated through the MetaGPT framework, which enables sequential coordination and reproducible communication among specialized agents for static analysis, refactoring strategy planning, and code...

[52] [Agentic Bug Reproduction for Effective Automated Program Repair at Google](#)

Runxiang Cheng, ..., and Satish Chandra. ArXiv, 2025. 6 citations.

0% Topic Match

Abstract: Bug reports often lack sufficient detail for developers to reproduce and fix the underlying defects. Bug Reproduction Tests (BRTs), tests that fail when the bug is present and pass when it has been resolved, are crucial for debugging, but they are rarely included in bug reports, both in open-source and in industrial settings. Thus, automatically generating BRTs from bug reports has the potential to accelerate the debugging process and lower time to repair. This paper investigates automated BRT generation within an industry setting, specifically at Google, focusing on the challenges of a large-scale, proprietary codebase and considering real-world industry bugs...

[53] [InfCode: Adversarial Iterative Refinement of Tests and Patches for Reliable Software Issue Resolution](#)

KeFan Li, ..., and Weifeng Lv. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large language models have advanced software engineering automation, yet resolving real-world software issues remains difficult because it requires repository-level reasoning, accurate diagnostics, and strong verification signals. Existing agent-based and pipeline-based methods often rely on insufficient tests, which can lead to patches that satisfy verification but fail to fix the underlying defect. We present InfCode, an adversarial multi-agent framework for automated repository-level issue resolution. InfCode iteratively refines both tests and patches through adversarial interaction between a Test Patch Generator and a Code Patch Generator, while a Selector agent identifies the most reliable fix. The framework runs inside a containerized environment that...

[54] [EXPERPAIR: Dual-Memory Enhanced LLM-based Repository-Level Program Repair](#)

Fangwen Mu, ..., and Qing Wang. ArXiv, 2025. 9 citations.

0% Topic Match

Abstract: Automatically repairing software issues remains a fundamental challenge at the intersection of software engineering and AI. Although recent advancements in Large Language Models (LLMs) have demonstrated potential for repository-level repair tasks, current methodologies exhibit two notable limitations: (1) they often address issues in isolation, neglecting to incorporate insights from previously resolved issues, and (2) they rely on static and rigid prompting strategies, which constrain their ability to generalize across diverse and evolving issue scenarios. Inspired by the dual memory systems of human cognition, where episodic and semantic memories work synergistically to support human reasoning and decision-making, we propose ExpeRepair, a...

[55] [A Study on Thinking Patterns of Large Reasoning Models in Code Generation](#)

Kevin Halim, ..., and Yang Liu. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Currently, many large language models (LLMs) are utilized for software engineering tasks such as code generation. The emergence of more advanced models known as large reasoning models (LRMs), such as OpenAI's o3, DeepSeek R1, and Qwen3. They have demonstrated the capability of performing multi-step reasoning. Despite the advancement in LRMs, little attention has been paid to systematically analyzing the reasoning patterns these models exhibit and how such patterns influence the generated code. This paper presents a comprehensive study aimed at investigating and uncovering the reasoning behavior of LRMs during code generation. We prompted several state-of-the-art LRMs of varying sizes with...

[56] [RepairAgent: An Autonomous, LLM-Based Agent for Program Repair](#)

Islam Bouzenia, ..., and Michael Pradel. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2024. 177 citations.

0% Topic Match

Abstract: Automated program repair has emerged as a powerful technique to mitigate the impact of software bugs on system reliability and user experience. This paper introduces Repair Agent, the first work to address the program repair challenge through an autonomous agent based on a large language model (LLM). Unlike existing deep learning-based approaches, which prompt a model with a fixed prompt or in a fixed feedback loop, our work treats the LLM as an agent capable of autonomously planning and executing actions to fix bugs by invoking suitable tools. Repair Agent freely interleaves gathering information about the bug, gathering repair ingredients...

[57] [Beyond Final Code: A Process-Oriented Error Analysis of Software Development Agents in Real-World GitHub Scenarios](#)

Zhi Chen, ..., and Lingxiao Jiang. ArXiv, 2025. 4 citations.

0% Topic Match

Abstract: AI-driven software development has rapidly advanced with the emergence of software development agents that leverage large language models (LLMs) to tackle complex, repository-level software engineering tasks. These agents go beyond just generation of final code; they engage in multi-step reasoning, utilize various tools for code modification and debugging, and interact with execution environments to diagnose and iteratively resolve issues. However, most existing evaluations focus primarily on static analyses of final code outputs, yielding limited insights into the agents'dynamic problem-solving processes. To fill this gap, we conduct an in-depth empirical study on 3,977 solving-phase trajectories and 3,931 testing-phase logs from 8...

[58] [Beyond pip Install: Evaluating LLM Agents for the Automated Installation of Python Projects](#)

Louis Milliken, ..., and Shin Yoo. 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2024. 8 citations.

0% Topic Match

Abstract: Many works have recently proposed the use of Large Language Model (LLM) based agents for performing 'repository level' tasks, loosely defined as a set of tasks whose scopes are greater than a single file. This has led to speculation that the orchestration of these repository-level tasks could lead to software engineering agents capable of performing almost independently of human intervention. However, of the suite of tasks that would need to be performed by this autonomous software engineering agent, we argue that one important task is missing, which is to fulfil project level dependency by installing other repositories. To investigate the...

[59] [Multi-Agent Debate Strategies to Enhance Requirements Engineering with Large Language Models](#)

Marc Oriol, ..., and Xavier Franch. 2025 IEEE 33rd International Requirements Engineering Conference (RE), 2025. 0 citations.

0% Topic Match

Abstract: Context: Large Language Model (LLM) agents are becoming widely used for various Requirements Engineering (RE) tasks. Research on improving their accuracy mainly focuses on prompt engineering, model fine-tuning, and retrieval augmented generation. However, these methods often treat models as isolated black boxes - relying on single-pass outputs without iterative refinement or collaboration, limiting robustness and adaptability. Objective: We propose that, just as human debates enhance accuracy and reduce bias in RE tasks by incorporating diverse perspectives, different LLM agents debating and collaborating may achieve similar improvements. Our goal is to investigate whether Multi-Agent Debate (MAD) strategies can enhance RE performance....

[60] [OMAC: A Broad Optimization Framework for LLM-Based Multi-Agent Collaboration](#)

Shijun Li, ..., and Joydeep Ghosh. ArXiv, 2025. 1 citations.

0% Topic Match

Abstract: Agents powered by advanced large language models (LLMs) have demonstrated impressive capabilities across diverse complex applications. Recently, Multi-Agent Systems (MAS), wherein multiple agents collaborate and communicate with each other, have exhibited enhanced capabilities in complex tasks, such as high-quality code generation and arithmetic reasoning. However, the development of such systems often relies on handcrafted methods, and the literature on systematic design and optimization of LLM-based MAS remains limited. In this work, we introduce OMAC, a general framework designed for holistic optimization of LLM-based MAS. Specifically, we identify five key optimization dimensions for MAS, encompassing both agent functionality and collaboration structure....

[61] [Software Development Using a Multi-agent Approach](#)

Juliana Gomes Ribeiro, ..., and C. G. Ralha. Anais do XIX Workshop-Escola de Sistemas de Agentes, seus Ambientes e Aplicações (WESAAC 2025), 2025. 0 citations.

0% Topic Match

Abstract: Software development involves complexities in requirements analysis, code generation, and continuous validation. This work presents a multi-agent system (MAS) designed to enhance the software engineering process by integrating Large Language Model (LLM) capabilities with human oversight. The MAS comprises five specialized agents that communicate through a shared message broker and maintain context in a persistent knowledge base. These agents autonomously handle tasks such as architectural planning, artifact generation, code review, and operational management, while human experts intervene to ensure accuracy and strategic alignment. The findings indicate that LLMs can impact software engineering, offering tools and techniques that automate tasks, enhance...

[62] [CigaR: Cost-efficient Program Repair with LLMs](#)

D'avid Hidv'egi, ..., and Monperrus Martin. ArXiv, 2024. 33 citations.

0% Topic Match

Abstract: Large language models (LLM) have proven to be effective at automated program repair (APR). However, using LLMs can be costly, with companies invoicing users by the number of tokens. In this paper, we propose CigaR, the first LLM-based APR tool that focuses on minimizing the repair cost. CigaR works in two major steps: generating a first plausible patch and multiplying plausible patches. CigaR optimizes the prompts and the prompt setting to maximize the information given to LLMs using the smallest possible number of tokens. Our experiments on 429 bugs from the widely used Defects4J and HumanEval-Java datasets shows that CigaR...

[63] [Self-Abstraction from Grounded Experience for Plan-Guided Policy Refinement](#)

Hiroaki Hayashi, ..., and Yingbo Zhou. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large language model (LLM) based agents are increasingly used to tackle software engineering tasks that require multi-step reasoning and code modification, demonstrating promising yet limited performance. However, most existing LLM agents typically operate within static execution frameworks, lacking a principled mechanism to learn and self-improve from their own experience and past rollouts. As a result, their performance remains bounded by the initial framework design and the underlying LLM's capabilities. We propose Self-Abstraction from Grounded Experience (SAGE), a framework that enables agents to learn from their own task executions and refine their behavior through self-abstraction. After an initial rollout, the agent...

[64] [Collaboration Dynamics and Reliability Challenges of Multi-Agent LLM Systems in Finite Element Analysis](#)

Chuan Tian and Yilei Zhang. ArXiv, 2024. 6 citations.

0% Topic Match

Abstract: Large Language Model (LLM)-based multi-agent systems are increasingly applied to automate computational workflows in science and engineering. However, how inter-agent dynamics influence reasoning quality and verification reliability remains unclear. We study these mechanisms using an AutoGen-based multi-agent framework for linear-elastic Finite Element Analysis (FEA), evaluating seven role configurations across four tasks under a fixed 12-turn conversation limit. From 1,120 controlled trials, we find that collaboration effectiveness depends more on functional complementarity than team size: the three-agent Coder-Executor-Critic configuration uniquely produced physically and visually correct solutions, while adding redundant reviewers reduced success rates. Yet three systematic failure modes persist: (1) affirmation...

[65] [Debug Smarter, Not Harder: AI Agents for Error Resolution in Computational Notebooks](#)

Konstantin Grotov, ..., and Yaroslav Zharov. ArXiv, 2024. 4 citations.

0% Topic Match

Abstract: Computational notebooks became indispensable tools for research-related development, offering unprecedented interactivity and flexibility in the development process. However, these benefits come at the cost of reproducibility and an increased potential for bugs. With the rise of code-fluent Large Language Models empowered with agentic techniques, smart bug-fixing tools with a high level of autonomy have emerged. However, those tools are tuned for classical script programming and still struggle with non-linear computational notebooks. In this paper, we present an AI agent designed specifically for error resolution in a computational notebook. We have developed an agentic system capable of exploring a notebook environment by interacting with...

[66] [Optima: Optimizing Effectiveness and Efficiency for LLM-Based Multi-Agent System](#)

Weize Chen, ..., and Maosong Sun. Unknown journal, 2024. 13 citations.

0% Topic Match

Abstract: Large Language Model (LLM) based multi-agent systems (MAS) show remarkable potential in collaborative problem-solving, yet they still face critical challenges: low communication efficiency, poor scalability, and a lack of effective parameter-updating optimization methods. We present Optima, a novel framework that addresses these issues by significantly enhancing both communication efficiency and task effectiveness in LLM-based MAS through LLM training. Optima employs an iterative generate, rank, select, and train paradigm with a reward function balancing task performance, token efficiency, and communication readability. We explore various RL algorithms, including Supervised Fine-Tuning, Direct Preference Optimization, and their hybrid approaches, providing insights into their effectiveness-efficiency...

[67] [Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis](#)

Yu Ge, ..., and Tian Zhang. ArXiv, 2025. 3 citations.

0% Topic Match

Abstract: Large Language Model Powered Multi-Agent Systems (MASs) are increasingly employed to automate complex real-world problems, such as programming and scientific discovery. Despite their promising, MASs are not without their flaws. However, failure attribution in MASs - pinpointing the specific agent actions responsible for failures - remains underexplored and labor-intensive, posing significant challenges for debugging and system improvement. To bridge this gap, we propose FAMAS, the first spectrum-based failure attribution approach for MASs, which operates through systematic trajectory replay and abstraction, followed by spectrum analysis. The core idea of FAMAS is to estimate, from variations across repeated MAS executions, the likelihood that...

[68] [Balancing Autonomy and Alignment: A Multi-Dimensional Taxonomy for Autonomous LLM-powered Multi-Agent Architectures](#)

Thorsten Händler. ArXiv, 2023. 32 citations.

0% Topic Match

Abstract: Large language models (LLMs) have revolutionized the field of artificial intelligence, endowing it with sophisticated language understanding and generation capabilities. However, when faced with more complex and interconnected tasks that demand a profound and iterative thought process, LLMs reveal their inherent limitations. Autonomous LLM-powered multi-agent systems represent a strategic response to these challenges. Such systems strive for autonomously tackling user-prompted goals by decomposing them into manageable tasks and orchestrating their execution and result synthesis through a collective of specialized intelligent agents. Equipped with LLM-powered reasoning capabilities, these agents harness the cognitive synergy of collaborating with their peers, enhanced by leveraging...

[69] [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#)

Carlos E. Jimenez, ..., and Karthik Narasimhan. ArXiv, 2023. 1117 citations.

0% Topic Match

Abstract: Language models have outpaced our ability to evaluate them effectively, but for their future development it is essential to study the frontier of their capabilities. We find real-world software engineering to be a rich, sustainable, and challenging testbed for evaluating the next generation of language models. To this end, we introduce SWE-bench, an evaluation framework consisting of \$2,294\$ software engineering problems drawn from real GitHub issues and corresponding pull requests across \$12\$ popular Python repositories. Given a codebase along with a description of an issue to be resolved, a language model is tasked with editing the codebase to address the...

[70] [RepoDebug: Repository-Level Multi-Task and Multi-Language Debugging Evaluation of Large Language Models](#)

Jingjing Liu, ..., and Haifeng Wang. ArXiv, 2025. 2 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have exhibited significant proficiency in code debugging, especially in automatic program repair, which may substantially reduce the time consumption of developers and enhance their efficiency. Significant advancements in debugging datasets have been made to promote the development of code debugging. However, these datasets primarily focus on assessing the LLM's function-level code repair capabilities, neglecting the more complex and realistic repository-level scenarios, which leads to an incomplete understanding of the LLM's challenges in repository-level debugging. While several repository-level datasets have been proposed, they often suffer from limitations such as limited diversity of tasks, languages, and error types...

[71] [MarsCode Agent: AI-native Automated Bug Fixing](#)

Yizhou Liu, ..., and Chao Peng. ArXiv, 2024. 64 citations.

0% Topic Match

Abstract: Recent advances in large language models (LLMs) have shown significant potential to automate various software development tasks, including code completion, test generation, and bug fixing. However, the application of LLMs for automated bug fixing remains challenging due to the complexity and diversity of real-world software systems. In this paper, we introduce MarsCode Agent, a novel framework that leverages LLMs to automatically identify and repair bugs in software code. MarsCode Agent combines the power of LLMs with advanced code analysis techniques to accurately localize faults and generate patches. Our approach follows a systematic process of planning, bug reproduction, fault localization, candidate...

[72] [CodePlan: Repository-Level Coding using LLMs and Planning](#)

Ramakrishna Bairi, ..., and Shashank Shet. Proceedings of the ACM on Software Engineering, 2023. 148 citations.

0% Topic Match

Abstract: Software engineering activities such as package migration, fixing error reports from static analysis or testing, and adding type annotations or other specifications to a codebase, involve pervasively editing the entire repository of code. We formulate these activities as repository-level coding tasks. Recent tools like GitHub Copilot, which are powered by Large Language Models (LLMs), have succeeded in offering high-quality solutions to localized coding problems. Repository-level coding tasks are more involved and cannot be solved directly using LLMs, since code within a repository is inter-dependent and the entire repository may be too large to fit into the prompt. We frame repository-level...

[73] [A Multi-Agent LLM Environment for Software Design and Refactoring: A Conceptual Framework](#)

Vasanth Rajendran, ..., and Vishnu Challagulla. SoutheastCon 2025, 2025. 1 citations.

0% Topic Match

Abstract: Modern software systems demand continuous evolution to maintain performance, scalability, and security. Traditional single-agent AI-driven code refactoring approaches are often limited in addressing the multi-faceted constraints (e.g., performance, security, maintainability) that emerge during complex software design tasks. In this paper, we propose a novel Multi-Agent Large Language Model (LLM) Environment for automated software design and refactoring. Our conceptual framework comprises specialized LLM "experts," each trained or fine-tuned on a different aspect of software engineering (performance optimization, security hardening, UI/UX, maintainability). These agents collaborate in a cooperative or competitive fashion using coordination protocols akin to consensus or auction mechanisms to synthesize design insights...

[74] [Using Role-Specialized LLM Agents for Workflow Execution in a Product Development Setting](#)

Maximilian Kretschmar, ..., and Kristin Paetzold-Byhain. 2025 9th International Conference on Inventive Systems and Control (ICISC), 2025. 0 citations.

0% Topic Match

Abstract: Many product development tasks require multi-step reasoning and decision-making that exceed the capabilities of Large Language Models (LLMs) operating in isolation. This research introduces a framework for constructing multi-agent systems (MAS) with LLMs for product development and presents an empirical study of its effectiveness. By decomposing workflows into subtasks handled by role-specialized agents, the system leverages agentic design principles and outperforms isolated model outputs. We evaluate this approach using LLaMA-3.1 (8B) and Gemma 2.0 (9B) in a realistic engineering scenario. While enabling more complex problem solving, the MAS also introduces challenges in transparency and fault tracing across input-output chains. Our...

[75] [Lingxi: Repository-Level Issue Resolution Framework Enhanced by Procedural Knowledge Guided Scaling](#)

Xu Yang, ..., and Ruiqi Pan. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Driven by the advancements of Large Language Models (LLMs), LLM-powered agents are making significant improvements in software engineering tasks, yet struggle with complex, repository-level issue resolution. Existing agent-based methods have two key limitations. First, they lack of procedural knowledge (i.e., how an issue is fixed step-by-step and rationales behind it) to learn and leverage for issue resolution. Second, they rely on massive computational power to blindly explore the solution space. % To address those limitations, we propose Lingxi, an issue resolution framework that leverages procedural knowledge extracted from historical issue-fixing data to guide agents in solving repository-level issues. OurTool first...

[76] [TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved?](#)

Toufique Ahmed, ..., and Saurabh Sinha. ArXiv, 2024. 19 citations.

0% Topic Match

Abstract: Test-driven development (TDD) is the practice of writing tests first and coding later, and the proponents of TDD expound its numerous benefits. For instance, given an issue on a source code repository, tests can clarify the desired behavior among stakeholders before anyone writes code for the agreed-upon fix. Although there has been a lot of work on automated test generation for the practice "write code first, test later", there has been little such automation for TDD. Ideally, tests for TDD should be fail-to-pass (i.e., fail before the issue is resolved and pass after) and have good adequacy with respect to covering...

[77] [Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \\$0.42 Each using ChatGPT](#)

Chun Xia and Lingming Zhang. Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023. 229 citations.

0% Topic Match

Abstract: Automated Program Repair (APR) aims to automatically generate patches for buggy programs. Traditional APR techniques suffer from a lack of patch variety as they rely heavily on handcrafted or mined bug fixing patterns and cannot easily generalize to other bug/fix types. To address this limitation, recent APR work has been focused on leveraging modern Large Language Models (LLMs) to directly generate patches for APR. Such LLM-based APR tools work by first constructing an input prompt built using the original buggy code and then querying the LLM to either fill-in (cloze-style APR) the correct code at the bug location or to...

[78] [RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation](#)

Fengji Zhang, ..., and Weizhu Chen. Unknown journal, 2023. 320 citations.

0% Topic Match

Abstract: The task of repository-level code completion is to continue writing the unfinished code based on a broader context of the repository. While for automated code completion tools, it is difficult to utilize the useful information scattered in different files. We propose RepoCoder, a simple, generic, and effective framework to address the challenge. It streamlines the repository-level code completion process by incorporating a similarity-based retriever and a pre-trained code language model in an iterative retrieval-generation pipeline. RepoCoder makes effective utilization of repository-level information for code completion and has the ability to generate code at various levels of granularity. Moreover, we propose...

[79] [Evaluating Large Language Models Trained on Code](#)

Mark Chen, ..., and Wojciech Zaremba. ArXiv, 2021. 7201 citations.

0% Topic Match

Abstract: We introduce Codex, a GPT language model fine-tuned on publicly available code from GitHub, and study its Python code-writing capabilities. A distinct production version of Codex powers GitHub Copilot. On HumanEval, a new evaluation set we release to measure functional correctness for synthesizing programs from docstrings, our model solves 28.8% of the problems, while GPT-3 solves 0% and GPT-J solves 11.4%. Furthermore, we find that repeated sampling from the model is a surprisingly effective strategy for producing working solutions to difficult prompts. Using this method, we solve 70.2% of our problems with 100 samples per problem. Careful investigation of our...

[80] [KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems](#)

Hancheng Ye, ..., and Yiran Chen. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Multi-agent large language model (LLM) systems are increasingly adopted for complex language processing tasks that require communication and coordination among agents. However, these systems often suffer substantial overhead from repeated reprocessing of overlapping contexts across agents. In typical pipelines, once an agent receives a message from its predecessor, the full context-including prior turns-must be reprocessed from scratch, leading to inefficient processing. While key-value (KV) caching is an effective solution for avoiding redundant computation in single-agent settings where prefixes remain unchanged, it cannot be directly reused in multi-agent scenarios due to diverging prefixes introduced by agent-specific context extensions. We identify that...

[81] [When "Correct" Is Not Safe: Can We Trust Functionally Correct Patches Generated by Code Agents?](#)

Yibo Peng, ..., and Beidi Chen. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Code agents are increasingly trusted to autonomously fix bugs on platforms such as GitHub, yet their security evaluation focuses almost exclusively on functional correctness. In this paper, we reveal a novel type of threat to real-world code agents: Functionally Correct yet Vulnerable (FCV) patches, which pass all test cases but contain vulnerable code. With our proposed FCV-Attack, which can be deliberately crafted by malicious attackers or implicitly introduced by benign developers, we show that SOTA LLMs (e.g., ChatGPT and Claude) and agent scaffolds (e.g., SWE-agent and OpenHands) are all vulnerable to this FCV threat; across 12 agent-model combinations on SWE-Bench,...

[82] [Specification and Evaluation of Multi-Agent LLM Systems - Prototype and Cybersecurity Applications](#)

Felix Härer. 2025 International Conference on Cybersecurity and AI-Based Systems (Cyber-AI), 2025. 2 citations.

0% Topic Match

Abstract: Recent advancements in LLMs indicate potential for novel applications, as evidenced by the reasoning capabilities in the latest OpenAI and DeepSeek models. To apply these models to domain-specific applications beyond text generation, LLM-based multi-agent systems can be utilized to solve complex tasks, particularly by combining reasoning techniques, code generation, and software execution across multiple, potentially specialized LLMs. However, while many evaluations are performed on LLMs, reasoning techniques, and applications individually, their joint specification and combined application are not well understood. Defined specifications for multi-agent LLM systems are required to explore their potential and suitability for specific applications, allowing for systematic...

[83] [Issue2Test: Generating Reproducing Test Cases from Issue Reports](#)

Noor Nashid, ..., and Ali Mesbah. ArXiv, 2025. 7 citations.

0% Topic Match

Abstract: Automated tools for solving GitHub issues are receiving significant attention by both researchers and practitioners, e.g., in the form of foundation models and LLM-based agents prompted with issues. A crucial step toward successfully solving an issue is creating a test case that accurately reproduces the issue. Such a test case can guide the search for an appropriate patch and help validate whether the patch matches the issue's intent. However, existing techniques for issue reproduction show only moderate success. This paper presents Issue2Test, an LLM-based technique for automatically generating a reproducing test case for a given issue report. Unlike automated regression...

[84] [ChatAssert: LLM-Based Test Oracle Generation With External Tools Assistance](#)

I. Hayet, ..., and Marcelo d'Amorim. IEEE Transactions on Software Engineering, 2025. 10 citations.

0% Topic Match

Abstract: Test oracle generation is an important and challenging problem. Neural-based solutions have been recently proposed for oracle generation but they are still inaccurate. For example, the accuracy of the state-of-the-art technique teco is only 27.5% on its dataset including 3,540 test cases. We propose ChatAssert, a prompt engineering framework designed for oracle generation that uses dynamic and static information to iteratively refine prompts for querying large language models (LLMs). ChatAssert uses code summaries and examples to assist an LLM in generating candidate test oracles, uses a lightweight static analysis to assist the LLM in repairing generated oracles that fail to...

[85] [RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems](#)

Tianyang Liu, ..., and Julian McAuley. ArXiv, 2023. 235 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have greatly advanced code auto-completion systems, with a potential for substantial productivity enhancements for developers. However, current benchmarks mainly focus on single-file tasks, leaving an assessment gap for more complex, real-world, multi-file programming scenarios. To fill this gap, we introduce RepoBench, a new benchmark specifically designed for evaluating repository-level code auto-completion systems. RepoBench supports both Python and Java and consists of three interconnected evaluation tasks: RepoBench-R (Retrieval), RepoBench-C (Code Completion), and RepoBench-P (Pipeline). Each task respectively measures the system's ability to retrieve the most relevant code snippets from other files as cross-file context, predict the next...

[86] [Demystifying Long Chain-of-Thought Reasoning in LLMs](#)

Edward Y. Chang, ..., and Xiang Yue. ArXiv, 2025. 238 citations.

0% Topic Match

Abstract: Scaling inference compute enhances reasoning in large language models (LLMs), with long chains-of-thought (CoTs) enabling strategies like backtracking and error correction. Reinforcement learning (RL) has emerged as a crucial method for developing these capabilities, yet the conditions under which long CoTs emerge remain unclear, and RL training requires careful design choices. In this study, we systematically investigate the mechanics of long CoT reasoning, identifying the key factors that enable models to generate long CoT trajectories. Through extensive supervised fine-tuning (SFT) and RL experiments, we present four main findings: (1) While SFT is not strictly necessary, it simplifies training and improves...

[87] [DARS: Dynamic Action Re-Sampling to Enhance Coding Agent Performance by Adaptive Tree Traversal](#)

Vaibhav Aggarwal, ..., and Bernhard Schölkopf. Unknown journal, 2025. 11 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have revolutionized various domains, including natural language processing, data analysis, and software development, by enabling automation. In software engineering, LLM-powered coding agents have garnered significant attention due to their potential to automate complex development tasks, assist in debugging, and enhance productivity. However, existing approaches often struggle with sub-optimal decision-making, requiring either extensive manual intervention or inefficient compute scaling strategies. To improve coding agent performance, we present Dynamic Action Re-Sampling (DARS), a novel inference time compute scaling approach for coding agents, that is faster and more effective at recovering from sub-optimal decisions compared to baselines. While traditional...

[88] [Benchmarking and Enhancing LLM Agents in Localizing Linux Kernel Bugs](#)

Zhenhao Zhou, ..., and Yiling Lou. ArXiv, 2025. 2 citations.

0% Topic Match

Abstract: The Linux kernel is a critical system, serving as the foundation for numerous systems. Bugs in the Linux kernel can cause serious consequences, affecting billions of users. Fault localization (FL), which aims at identifying the buggy code elements in software, plays an essential role in software quality assurance. While recent LLM agents have achieved promising accuracy in FL on recent benchmarks like SWE-bench, it remains unclear how well these methods perform in the Linux kernel, where FL is much more challenging due to the large-scale code base, limited observability, and diverse impact factors. In this paper, we introduce LinuxFLBench, a...

[89] [Evaluating and Improving ChatGPT for Unit Test Generation](#)

Zhiqiang Yuan, ..., and Yiling Lou. Proceedings of the ACM on Software Engineering, 2024. 107 citations.

0% Topic Match

Abstract: Unit testing plays an essential role in detecting bugs in functionally-discrete program units (e.g., methods). Manually writing high-quality unit tests is time-consuming and laborious. Although the traditional techniques are able to generate tests with reasonable coverage, they are shown to exhibit low readability and still cannot be directly adopted by developers in practice. Recent work has shown the large potential of large language models (LLMs) in unit test generation. By being pre-trained on a massive developer-written code corpus, the models are capable of generating more human-like and meaningful test code. In this work, we perform the first empirical study to...

[90] [You Name It, I Run It: An LLM Agent to Execute Tests of Arbitrary Projects](#)

Islem Bouzenia and Michael Pradel. Proceedings of the ACM on Software Engineering, 2024. 24 citations.

0% Topic Match

Abstract: The ability to execute the test suite of a project is essential in many scenarios, e.g., to assess code quality and code coverage, to validate code changes made by developers or automated tools, and to ensure compatibility with dependencies. Despite its importance, executing the test suite of a project can be challenging in practice because different projects use different programming languages, software ecosystems, build systems, testing frameworks, and other tools. These challenges make it difficult to create a reliable, universal test execution method that works across different projects. This paper presents ExecutionAgent, an automated technique that prepares scripts for building...

[91] [EvoCodeBench: An Evolving Code Generation Benchmark with Domain-Specific Evaluations](#)

Jia Li, ..., and Yongbin Li. ArXiv, 2024. 30 citations.

0% Topic Match

Abstract: How to evaluate Large Language Models (LLMs) in code generation remains an open question. Existing benchmarks have two limitations - data leakage and lack of domain-specific evaluation. The former hurts the fairness of benchmarks, and the latter hinders practitioners from selecting superior LLMs for specific programming domains. To address these two limitations, we propose a new benchmark - EvoCodeBench, which has the following advances: (1) Evolving data. EvoCodeBench will be dynamically updated every period (e.g., 6 months) to avoid data leakage. This paper releases the first version - EvoCodeBench-2403, containing 275 samples from 25 repositories. (2) A domain taxonomy and...

[92] [Evaluation of Different Large Language Model Agent Frameworks for Design Engineering Tasks](#)

Alejandro Pradas Gómez, ..., and Ola Isaksson. Proceedings of NordDesign 2024, 2024. 5 citations.

0% Topic Match

Abstract: : This paper evaluates Large Language Models (LLMs) ability to support engineering tasks. Reasoning frameworks such as agents and multi-agents are described and compared. The frameworks are implemented with the LangChain python package for an engineering task. The results show that a supportive reasoning framework can increase the quality of responses compared to a standalone LLM. Their applicability to other engineering tasks is discussed. Finally, a perspective of task ownership is presented between the designer, the traditional software, and the Generative AI.

[93] [MAGDi: Structured Distillation of Multi-Agent Interaction Graphs Improves Reasoning in Smaller Language Models](#)

Justin Chih-Yao Chen, ..., and Mohit Bansal. ArXiv, 2024. 26 citations.

0% Topic Match

Abstract: Multi-agent interactions between Large Language Model (LLM) agents have shown major improvements on diverse reasoning tasks. However, these involve long generations from multiple models across several rounds, making them expensive. Moreover, these multi-agent approaches fail to provide a final, single model for efficient inference. To address this, we introduce MAGDi, a new method for structured distillation of the reasoning interactions between multiple LLMs into smaller LMs. MAGDi teaches smaller models by representing multi-agent interactions as graphs, augmenting a base student model with a graph encoder, and distilling knowledge using three objective functions: next-token prediction, a contrastive loss between correct and...

[94] [On the Resilience of LLM-Based Multi-Agent Collaboration with Faulty Agents](#)

Jen-Tse Huang, ..., and Michael R. Lyu. ArXiv, 2024. 25 citations.

0% Topic Match

Abstract: Large language model-based multi-agent systems have shown great abilities across various tasks due to the collaboration of expert agents, each focusing on a specific domain. However, the impact of clumsy or even malicious agents--those who frequently make errors in their tasks--on the overall performance of the system remains underexplored. This paper investigates: (1) What is the resilience of various system structures (e.g., $A \rightarrow B \rightarrow C$, $A \rightarrow B \rightarrow C$, $A \rightarrow B \rightarrow C$) under faulty agents, on different downstream tasks? (2) How can we increase system resilience to defend against these agents? To simulate faulty agents, we propose two approaches--AutoTransform and AutoInject--which introduce mistakes into the agents' responses. Experiments...

[95] [Calibration and Correctness of Language Models for Code](#)

Claudio Spiess, ..., and Toufique Ahmed. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2024. 48 citations.

0% Topic Match

Abstract: Machine learning models are widely used, but can also often be wrong. Users would benefit from a reliable indication of whether a given output from a given model should be trusted, so a rational decision can be made whether to use the output or not. For example, outputs can be associated with a confidence measure; if this confidence measure is strongly associated with likelihood of correctness, then the model is said to be well-calibrated. A well-calibrated confidence measure can serve as a basis for rational, graduated decision-making on how much review and care is needed when using generated code. Calibration...

[96] [SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering](#)

John Yang, ..., and Ofir Press. ArXiv, 2024. 550 citations.

0% Topic Match

Abstract: Language model (LM) agents are increasingly being used to automate complicated tasks in digital environments. Just as humans benefit from powerful software applications, such as integrated development environments, for complex tasks like software engineering, we posit that LM agents represent a new category of end users with their own needs and abilities, and would benefit from specially-built interfaces to the software they use. We investigate how interface design affects the performance of language model agents. As a result of this exploration, we introduce SWE-agent: a system that facilitates LM agents to autonomously use computers to solve software engineering tasks. SWE-agent's...

[97] [Traceability and Accountability in Role-Specialized Multi-Agent LLM Pipelines](#)

Amine Barrak. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Sequential multi-agent systems built with large language models (LLMs) can automate complex software tasks, but they are hard to trust because errors quietly pass from one stage to the next. We study a traceable and accountable pipeline, meaning a system with clear roles, structured handoffs, and saved records that let us trace who did what at each step and assign blame when things go wrong. Our setting is a Planner ->Executor ->Critic pipeline. We evaluate eight configurations of three state-of-the-art LLMs on three benchmarks and analyze where errors start, how they spread, and how they can be fixed. Our results...

[98] [Refactoring Codebases through Library Design](#)

Ziga Kovacic, ..., and Kevin Ellis. ArXiv, 2025. 1 citations.

0% Topic Match

Abstract: Maintainable and general software allows developers to build robust applications efficiently, yet achieving these qualities often requires refactoring specialized solutions into reusable components. This challenge becomes particularly relevant as code agents become used to solve isolated one-off programming problems. We investigate code agents' capacity to refactor code in ways that support growth and reusability. We first investigate what makes a good refactoring, finding via simulation results and a human study that Minimum Description Length best correlates with preferable refactorings. We then present both a benchmark and a method for refactoring: MiniCode, a benchmark where multiple files must be refactored into a...

[99] [ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate](#)

Chi-Min Chan, ..., and Zhiyuan Liu. ArXiv, 2023. 675 citations.

0% Topic Match

Abstract: Text evaluation has historically posed significant challenges, often demanding substantial labor and time cost. With the emergence of large language models (LLMs), researchers have explored LLMs' potential as alternatives for human evaluation. While these single-agent-based approaches show promise, experimental results suggest that further advancements are needed to bridge the gap between their current effectiveness and human-level evaluation quality. Recognizing that best practices of human evaluation processes often involve multiple human annotators collaborating in the evaluation, we resort to a multi-agent debate framework, moving beyond single-agent prompting strategies. The multi-agent-based approach enables a group of LLMs to synergize with an array...

[100] [LLM-Powered Multi-Agent Systems: A Technical Framework for Collaborative Intelligence Through Optimized Knowledge Retrieval and Communication](#)

Vineeth Gogineni. 2025 6th International Conference on Artificial Intelligence, Robotics and Control (AIRC), 2025. 0 citations.

0% Topic Match

Abstract: This paper presents a comprehensive technical framework for constructing effective multi-agent systems powered by large language models (LLMs). We examine how multiple LLM-based agents can collaborate to solve complex problems beyond the capabilities of single agents through specialized knowledge integration and optimized communication protocols. Our novel architecture enables efficient collaboration between heterogeneous LLM agents, each with domain-specific capabilities and knowledge bases. Experimental results demonstrate that our multi-agent LLM system achieves 42% higher accuracy on complex knowledge tasks, 37% reduction in hallucinations, and 29% faster convergence on collaborative problem-solving compared to baseline approaches. By implementing optimized knowledge retrieval mechanisms and structured...

[101] [OPTAGENT: Optimizing Multi-Agent LLM Interactions Through Verbal Reinforcement Learning for Enhanced Reasoning](#)

Zhenyu Bi, ..., and Xuan Wang. ArXiv, 2025. 1 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have shown remarkable reasoning capabilities in mathematical and scientific tasks. To enhance complex reasoning, multi-agent systems have been proposed to harness the collective intelligence of LLM agents. However, existing collaboration structures are either predefined or rely on majority voting or round-table debates, which can suppress correct but less dominant agent contributions. Recent approaches model multi-agent systems as graph networks but optimize purely for agent performance, neglecting the quality of interactions. We hypothesize that effective agent communication is crucial for multi-agent reasoning and that debating quality plays a significant role. To address this, we propose \$lours\$, a...

[102] [CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing](#)

Zhibin Gou, ..., and Weizhu Chen. ArXiv, 2023. 526 citations.

0% Topic Match

Abstract: Recent developments in large language models (LLMs) have been impressive. However, these models sometimes show inconsistencies and problematic behavior, such as hallucinating facts, generating flawed code, or creating offensive and toxic content. Unlike these models, humans typically utilize external tools to cross-check and refine their initial content, like using a search engine for fact-checking, or a code interpreter for debugging. Inspired by this observation, we introduce a framework called CRITIC that allows LLMs, which are essentially "black boxes" to validate and progressively amend their own outputs in a manner similar to human interaction with tools. More specifically, starting with an initial output,...

[103] [TOGLL: Correct and Strong Test Oracle Generation with LLMs](#)

Soneya Binta Hossain and Matthew B. Dwyer. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2024. 30 citations.

0% Topic Match

Abstract: Test oracles play a crucial role in software testing, enabling effective bug detection. Despite initial promise, neural methods for automated test oracle generation often result in a large number of false positives and weaker test oracles. While LLMs have shown impressive effectiveness in various software engineering tasks, including code generation, test case creation, and bug fixing, there remains a notable absence of large-scale studies exploring their effectiveness in test oracle generation. The question of whether LLMs can address the challenges in effective oracle generation is both compelling and requires thorough investigation. In this research, we present the first comprehensive study...

[104] [Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly?](#)

Chunqiu Steven Xia, ..., and Ling-kai Zhang. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) are reshaping almost all industries, including software engineering. In recent years, a number of LLM agents have been proposed to solve real-world software problems. Such software agents are typically equipped with a suite of coding tools and can autonomously decide the next actions to form complete trajectories to solve end-to-end software tasks. While promising, they typically require dedicated design and may still be suboptimal, since it can be extremely challenging and costly to exhaust the entire agent scaffold design space. Recognizing that software agents are inherently software themselves that can be further refined/modified, researchers have proposed...

[105] [SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents](#)

Niels Mündler, ..., and Martin T. Vechev. Advances in Neural Information Processing Systems 37, 2024. 61 citations.

0% Topic Match

Abstract: Rigorous software testing is crucial for developing and maintaining high-quality code, making automated test generation a promising avenue for both improving software quality and boosting the effectiveness of code generation methods. However, while code generation with Large Language Models (LLMs) is an extraordinarily active research area, test generation remains relatively unexplored. We address this gap and investigate the capability of LLM-based Code Agents to formalize user issues into test cases. To this end, we propose a novel benchmark based on popular GitHub repositories, containing real-world issues, ground-truth bug-fixes, and golden tests. We find that LLMs generally perform surprisingly well at...

[106] [AgentBank: Towards Generalized LLM Agents via Fine-Tuning on 50000+ Interaction Trajectories](#)

Yifan Song, ..., and Sujian Li. Unknown journal, 2024. 24 citations.

0% Topic Match

Abstract: Fine-tuning on agent-environment interaction trajectory data holds significant promise for surfacing generalized agent capabilities in open-source large language models (LLMs). In this work, we introduce AgentBank, by far the largest trajectory tuning data collection featuring more than 50k diverse high-quality interaction trajectories which comprises 16 tasks covering five distinct agent skill dimensions. Leveraging a novel annotation pipeline, we are able to scale the annotated trajectories and generate a trajectory dataset with minimized difficulty bias. Furthermore, we fine-tune LLMs on AgentBank to get a series of agent models, Samoyed. Our comparative experiments demonstrate the effectiveness of scaling the interaction trajectory data...

[107] [Can Large Language Models Really Improve by Self-critiquing Their Own Plans?](#)

Karthik Valmeekam, ..., and Subbarao Kambhampati. ArXiv, 2023. 110 citations.

0% Topic Match

Abstract: There have been widespread claims about Large Language Models (LLMs) being able to successfully verify or self-critique their candidate solutions in reasoning problems in an iterative mode. Intrigued by those claims, in this paper we set out to investigate the verification/self-critiquing abilities of large language models in the context of planning. We evaluate a planning system that employs LLMs for both plan generation and verification. We assess the verifier LLM's performance against ground-truth verification, the impact of self-critiquing on plan generation, and the influence of varying feedback levels on system performance. Using GPT-4, a state-of-the-art LLM, for both generation and...

[108] [S2-MAD: Breaking the Token Barrier to Enhance Multi-Agent Debate Efficiency](#)

Yuting Zeng, ..., and Xiaohua Xu. ArXiv, 2025. 7 citations.

0% Topic Match

Abstract: Large language models (LLMs) have demonstrated remarkable capabilities across various natural language processing (NLP) scenarios, but they still face challenges when handling complex arithmetic and logical reasoning tasks. While Chain-Of-Thought (CoT) reasoning, self-consistency (SC) and self-correction strategies have attempted to guide models in sequential, multi-step reasoning, Multi-agent Debate (MAD) has emerged as a viable approach for enhancing the reasoning capabilities of LLMs. By increasing both the number of agents and the frequency of debates, the performance of LLMs improves significantly. However, this strategy results in a significant increase in token costs, presenting a barrier to scalability. To address this challenge,...

[109] [On the Importance of Task Complexity in Evaluating LLM-Based Multi-Agent Systems](#)

Bohan Tang, ..., and Xiaowen Dong. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Large language model multi-agent systems (LLM-MAS) offer a promising paradigm for harnessing collective intelligence to achieve more advanced forms of AI behaviour. While recent studies suggest that LLM-MAS can outperform LLM single-agent systems (LLM-SAS) on certain tasks, the lack of systematic experimental designs limits the strength and generality of these conclusions. We argue that a principled understanding of task complexity, such as the degree of sequential reasoning required and the breadth of capabilities involved, is essential for assessing the effectiveness of LLM-MAS in task solving. To this end, we propose a theoretical framework characterising tasks along two dimensions: depth, representing...

[110] [Fuzz4ALL: Universal Fuzzing with Large Language Models](#)

Chun Xia, ..., and Lingming Zhang. 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE), 2023. 207 citations.

0% Topic Match

Abstract: Fuzzing has achieved tremendous success in discovering bugs and vulnerabilities in various software systems. Systems under test (SUTs) that take in programming or formal language as inputs, e.g., compilers, runtime engines, constraint solvers, and software libraries with accessible APIs, are especially important as they are fundamental building blocks of software development. However, existing fuzzers for such systems often target a specific language, and thus cannot be easily applied to other languages or even other versions of the same language. Moreover, the inputs generated by existing fuzzers are often limited to specific features of the input language, and thus can hardly...

[111] [CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis](#)

Erik Nijkamp, ..., and Caiming Xiong. International Conference on Learning Representations, 2022. 1284 citations.

0% Topic Match

Abstract: Program synthesis strives to generate a computer program as a solution to a given problem specification, expressed with input-output examples or natural language descriptions. The prevalence of large language models advances the state-of-the-art for program synthesis, though limited training resources and data impede open access to such models. To democratize this, we train and release a family of large language models up to 16.1B parameters, called CODEGEN, on natural language and programming language data, and open source the training library JAXFORMER. We show the utility of the trained model by demonstrating that it is competitive with the previous state-of-the-art on...

[112] [Research on the role of LLM in multi-agent systems: A survey](#)

Jianxiang Ma. Applied and Computational Engineering, 2024. 2 citations.

0% Topic Match

Abstract: In recent years, the rapid development of large language model (LLM) has demonstrated superior performance in language understanding, text generation, planning, reasoning, and knowledge integration. This has led to the emergence of intelligent agents based on LLM. By leveraging the capabilities of LLM, these agents can effectively make decisions based on given objectives and possess certain learning and adaptation abilities. However, single-agent systems are generally suited to solving relatively simple problems and are limited in handling complex tasks that require coordination. For instance, in fields such as power grid management or traffic control systems, relying solely on a single agent...

[113] [Debating with More Persuasive LLMs Leads to More Truthful Answers](#)

Akbir Khan, ..., and Ethan Perez. ArXiv, 2024. 185 citations.

0% Topic Match

Abstract: Common methods for aligning large language models (LLMs) with desired behaviour heavily rely on human-labelled data. However, as models grow increasingly sophisticated, they will surpass human expertise, and the role of human evaluation will evolve into non-experts overseeing experts. In anticipation of this, we ask: can weaker models assess the correctness of stronger models? We investigate this question in an analogous setting, where stronger models (experts) possess the necessary information to answer questions and weaker models (non-experts) lack this information. The method we evaluate is debate, where two LLM experts each argue for a different answer, and a non-expert selects...

[114] [Exploring LLM-Based Agents for Root Cause Analysis](#)

Devjeet Roy, ..., and S. Rajmohan. Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, 2024. 62 citations.

0% Topic Match

Abstract: The growing complexity of cloud based software systems has resulted in incident management becoming an integral part of the software development lifecycle. Root cause analysis (RCA), a critical part of the incident management process, is a demanding task for on-call engineers, requiring deep domain knowledge and extensive experience with a team's specific services. Automation of RCA can result in significant savings of time, and ease the burden of incident management on on-call engineers. Recently, researchers have utilized Large Language Models (LLMs) to perform RCA, and have demonstrated promising results. However, these approaches are not able to dynamically collect additional diagnostic...

[115] [Can Lessons From Human Teams Be Applied to Multi-Agent Systems? The Role of Structure, Diversity, and Interaction Dynamics](#)

Rasika Muralidharan, ..., and Jisun An. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Multi-Agent Systems (MAS) with Large Language Model (LLM)-powered agents are gaining attention, yet fewer studies explore their team dynamics. Inspired by human team science, we propose a multi-agent framework to examine core aspects of team science: structure, diversity, and interaction dynamics. We evaluate team performance across four tasks: CommonsenseQA, StrategyQA, Social IQa, and Latent Implicit Hate, spanning commonsense and social reasoning. Our results show that flat teams tend to perform better than hierarchical ones, while diversity has a nuanced impact. Interviews suggest agents are overconfident about their team performance, yet post-task reflections reveal both appreciation for collaboration and challenges in...

[116] [Neural software analysis](#)

Michael Pradel and S. Chandra. Communications of the ACM, 2020. 35 citations.

0% Topic Match

Abstract: Developer tools that use a neural machine learning model to make predictions about previously unseen code.

[117] [StarCoder: may the source be with you!](#)

Raymond Li, ..., and H. D. Vries. Trans. Mach. Learn. Res., 2023. 984 citations.

0% Topic Match

Abstract: The BigCode community, an open-scientific collaboration working on the responsible development of Large Language Models for Code (Code LLMs), introduces StarCoder and StarCoderBase: 15.5B parameter models with 8K context length, infilling capabilities and fast large-batch inference enabled by multi-query attention. StarCoderBase is trained on 1 trillion tokens sourced from The Stack, a large collection of permissively licensed GitHub repositories with inspection tools and an opt-out process. We fine-tuned StarCoderBase on 35B Python tokens, resulting in the creation of StarCoder. We perform the most comprehensive evaluation of Code LLMs to date and show that StarCoderBase outperforms every open Code LLM that...

[118] [Unleashing Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration](#)

Zhenhailong Wang, ..., and Heng Ji. ArXiv, 2023. 100 citations.

0% Topic Match

No summary or abstract available

[119] [Revisiting Multi-Agent Debate as Test-Time Scaling: A Systematic Study of Conditional Effectiveness](#)

Yongjin Yang, ..., and SeYoung Yun. ArXiv, 2025. 8 citations.

0% Topic Match

Abstract: The remarkable growth in large language model (LLM) capabilities has spurred exploration into multi-agent systems, with debate frameworks emerging as a promising avenue for enhanced problem-solving. These multi-agent debate (MAD) approaches, where agents collaboratively present, critique, and refine arguments, potentially offer improved reasoning, robustness, and diverse perspectives over monolithic models. Despite prior studies leveraging MAD, a systematic understanding of its effectiveness compared to self-agent methods, particularly under varying conditions, remains elusive. This paper seeks to fill this gap by conceptualizing MAD as a test-time computational scaling technique, distinguished by collaborative refinement and diverse exploration capabilities. We conduct a comprehensive empirical...

[120] [Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate](#)

Tian Liang, ..., and Shuming Shi. Unknown journal, 2023. 726 citations.

0% Topic Match

Abstract: Modern large language models (LLMs) like ChatGPT have shown remarkable performance on general language tasks but still struggle on complex reasoning tasks, which drives the research on cognitive behaviors of LLMs to explore human-like problem-solving strategies. Along this direction, one representative strategy is self-reflection, which asks an LLM to refine the solution with the feedback generated by itself iteratively. However, our study shows that such reflection-style methods suffer from the Degeneration-of-Thought (DoT) problem: once the LLM has established confidence in its solutions, it is unable to generate novel thoughts later through reflection even if its initial stance is incorrect. To...

[121] [AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents](#)

Chang Ma, ..., and Junxian He. ArXiv, 2024. 120 citations.

0% Topic Match

Abstract: Evaluating Large Language Models (LLMs) as general-purpose agents is essential for understanding their capabilities and facilitating their integration into practical applications. However, the evaluation process presents substantial challenges. A primary obstacle is the benchmarking of agent performance across diverse scenarios within a unified framework, especially in maintaining partially-observable environments and ensuring multi-round interactions. Moreover, current evaluation frameworks mostly focus on the final success rate, revealing few insights during the process and failing to provide a deep understanding of the model abilities. To address these challenges, we introduce AgentBoard, a pioneering comprehensive benchmark and accompanied open-source evaluation framework tailored to analytical...

[122] [Trial and Error: Exploration-Based Trajectory Optimization for LLM Agents](#)

Yifan Song, ..., and Bill Yuchen Lin. ArXiv, 2024. 125 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have become integral components in various autonomous agent systems. In this study, we present an exploration-based trajectory optimization approach, referred to as ETO. This learning method is designed to enhance the performance of open LLM agents. Contrary to previous studies that exclusively train on successful expert trajectories, our method allows agents to learn from their exploration failures. This leads to improved performance through an iterative optimization framework. During the exploration phase, the agent interacts with the environment while completing given tasks, gathering failure trajectories to create contrastive trajectory pairs. In the subsequent training phase, the agent...

[123] [Coalitions of Large Language Models Increase the Robustness of AI Agents](#)

Prattyush Mangal, ..., and Edward Pyzer-Knapp. ArXiv, 2024. 1 citations.

0% Topic Match

Abstract: The emergence of Large Language Models (LLMs) have fundamentally altered the way we interact with digital systems and have led to the pursuit of LLM powered AI agents to assist in daily workflows. LLMs, whilst powerful and capable of demonstrating some emergent properties, are not logical reasoners and often struggle to perform well at all sub-tasks carried out by an AI agent to plan and execute a workflow. While existing studies tackle this lack of proficiency by generalised pretraining at a huge scale or by specialised fine-tuning for tool use, we assess if a system comprising of a coalition of...

[124] [VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks](#)

Jing Yu Koh, ..., and Daniel Fried. ArXiv, 2024. 330 citations.

0% Topic Match

No summary or abstract available

[125] [A 2030 Roadmap for Software Engineering](#)

Mauro Pezzè, ..., and Tao Yue. ACM Transactions on Software Engineering and Methodology, 2025. 4 citations.

0% Topic Match

Abstract: The landscape of software engineering has dramatically changed in recent years. The impressive advances of artificial intelligence are just the latest and most disruptive innovation that has remarkably changed the software engineering research and practice. This special issue shares a roadmap to guide the software engineering community in this confused era. This roadmap is the outcome of a 2-day intensive discussion at the 2030 Software Engineering workshop. The roadmap spotlights and discusses seven main landmarks in the new software engineering landscape: artificial intelligence for software engineering, human aspects of software engineering, software security, verification and validation, sustainable software engineering, automatic...

[126] [AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems](#)

Jiaying Lu, ..., and Wei Chen. IEEE Transactions on Visualization and Computer Graphics, 2024. 22 citations.

0% Topic Match

Abstract: Recently, Large Language Model based Autonomous System (LLMAS) has gained great popularity for its potential to simulate complicated behaviors of human societies. One of its main challenges is to present and analyze the dynamic events evolution of LLMAS. In this work, we present a visualization approach to explore the detailed statuses and agents' behavior within LLMAS. Our approach outlines a general pipeline that organizes raw execution events from LLMAS into a structured behavior model. We leverage a behavior summarization algorithm to create a hierarchical summary of these behaviors, arranged according to their sequence over time. Additionally, we design a cause...

[127] [On Generalization in Agentic Tool Calling: CoreThink Agentic Reasoner and MAVEN Dataset](#)

Vishvesh Bhat, ..., and Julian McAuley. ArXiv, 2025. 0 citations.

0% Topic Match

Abstract: Generalization across Agentic tool-calling environments remains a key unsolved challenge in developing reliable agentic reasoning systems. While large language models (LLMs) demonstrate strong performance on isolated benchmarks, their ability to transfer reasoning strategies and co-ordinate tools across diverse domains is poorly understood. In this work, we conduct a large-scale evaluation of state-of-the-art LLMs on multiple tool-calling benchmarks BFCL v3, TauBench, Tau2Bench, and AceBench and introduce MAVEN (Math&Physics Adversarial Verification&Evaluation Network), a new out of distribution (OOD) benchmark designed to stress-test multi-step reasoning through explicit verification and adversarial task composition. Our results show that most current models achieve below 50% accuracy on MAVEN...

[128] [MultiAgent Collaboration Attack: Investigating Adversarial Attacks in Large Language Model Collaborations via Debate](#)

Alfonso Amayuelas, ..., and W. Wang. Unknown journal, 2024. 28 citations.

0% Topic Match

Abstract: Large Language Models (LLMs) have shown exceptional results on current benchmarks when working individually. The advancement in their capabilities, along with a reduction in parameter size and inference times, has facilitated the use of these models as agents, enabling interactions among multiple models to execute complex tasks. Such collaborations offer several advantages, including the use of specialized models (e.g. coding), improved confidence through multiple computations, and enhanced divergent thinking, leading to more diverse outputs. Thus, the collaborative use of language models is expected to grow significantly in the coming years. In this work, we evaluate the behavior of a network...

[129] [From Novice to Expert: LLM Agent Policy Optimization via Step-wise Reinforcement Learning](#)

Zhirui Deng, ..., and Weipeng Chen. ArXiv, 2024. 15 citations.

0% Topic Match

Abstract: The outstanding capabilities of large language models (LLMs) render them a crucial component in various autonomous agent systems. While traditional methods depend on the inherent knowledge of LLMs without fine-tuning, more recent approaches have shifted toward the reinforcement learning strategy to further enhance agents' ability to solve complex interactive tasks with environments and tools. However, previous approaches are constrained by the sparse reward issue, where existing datasets solely provide a final scalar reward for each multi-step reasoning chain, potentially leading to ineffectiveness and inefficiency in policy learning. In this paper, we introduce StepAgent, which utilizes step-wise reward to optimize the...

[130] [MAMM-Refine: A Recipe for Improving Faithfulness in Generation with Multi-Agent Collaboration](#)

David Wan, ..., and Mohit Bansal. ArXiv, 2025. 4 citations.

0% Topic Match

Abstract: Multi-agent collaboration among models has shown promise in reasoning tasks but is underexplored in long-form generation tasks like summarization and question-answering. We extend multi-agent multi-model reasoning to generation, specifically to improving faithfulness through refinement, i.e., revising model-generated outputs to remove factual inconsistencies. We investigate how iterative collaboration among multiple instances and types of large language models (LLMs) enhances subtasks in the refinement process, such as error detection, critiquing unfaithful sentences, and making corrections based on critiques. We design intrinsic evaluations for each subtask, with our findings indicating that both multi-agent (multiple instances) and multi-model (diverse LLM types) approaches benefit error...

[131] [Defects4J: a database of existing faults to enable controlled testing studies for Java programs](#)

René Just, ..., and Michael D. Ernst. Unknown journal, 2014. 1452 citations.

0% Topic Match

No summary or abstract available

[132] [WebArena: A Realistic Web Environment for Building Autonomous Agents](#)

Shuyan Zhou, ..., and Graham Neubig. ArXiv, 2023. 740 citations.

0% Topic Match

Abstract: With advances in generative AI, there is now potential for autonomous agents to manage daily tasks via natural language commands. However, current agents are primarily created and tested in simplified synthetic environments, leading to a disconnect with real-world scenarios. In this paper, we build an environment for language-guided agents that is highly realistic and reproducible. Specifically, we focus on agents that perform tasks on the web, and create an environment with fully functional websites from four common domains: e-commerce, social forum discussions, collaborative software development, and content management. Our environment is enriched with tools (e.g., a map) and external knowledge...

- [133] [Methodology](#)
M. Salah and Fathi Abid. Child & Youth Services, 2003. 2935 citations.
0% Topic Match
No summary or abstract available
- [134] [Generative AI Efficiency and Effectiveness in Software Project Documentation Review Process](#)
Kadir Alpaslan Demir, ..., and Bozhen Liu. 2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA), 2025. 0 citations.
0% Topic Match
Abstract: In this research, we report the capabilities of Large Language Model (LLM)-based Generative Artificial Intelligence (GenAI) tools, such as ChatGPT and Gemini, in the software project documentation review process. In software projects, the document artifact review process is an essential part of the quality assurance process. This review process is a human effort-intensive process. Introducing automation in this process will help reduce human effort, speed up the development, and reduce the potential of errors in the documents. With its enhanced natural language processing capabilities, LLM-based GenAI tools provide significant potential in the above process. In this research, we investigate the...
- [135] [An LLM-based Agent for Reliable Docker Environment Configuration](#)
Ruida Hu, ..., and Cuiyun Gao. ArXiv, 2025. 10 citations.
0% Topic Match
No summary or abstract available
- [136] [Why Programs Fail: A Guide to Systematic Debugging](#)
A. Zeller. Unknown journal, 2005. 497 citations.
0% Topic Match
No summary or abstract available
- [137] [Beyond Single Models: Enhancing LLM Detection of Ambiguity in Requests through Debate](#)
Ana Davila, ..., and Yasuhisa Hasegawa. ArXiv, 2025. 2 citations.
0% Topic Match
Abstract: Large Language Models (LLMs) have demonstrated significant capabilities in understanding and generating human language, contributing to more natural interactions with complex systems. However, they face challenges such as ambiguity in user requests processed by LLMs. To address these challenges, this paper introduces and evaluates a multi-agent debate framework designed to enhance detection and resolution capabilities beyond single models. The framework consists of three LLM architectures (Llama3-8B, Gemma2-9B, and Mistral-7B variants) and a dataset with diverse ambiguities. The debate framework markedly enhanced the performance of Llama3-8B and Mistral-7B variants over their individual baselines, with Mistral-7B-led debates achieving a notable 76.7% success...
- [138] [WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents](#)
Shunyu Yao, ..., and Karthik Narasimhan. ArXiv, 2022. 704 citations.
0% Topic Match
Abstract: Existing benchmarks for grounding language in interactive environments either lack real-world linguistic elements, or prove difficult to scale up due to substantial human involvement in the collection of data or feedback signals. To bridge this gap, we develop WebShop -- a simulated e-commerce website environment with \$1.18\$ million real-world products and \$12,087\$ crowd-sourced text instructions. Given a text instruction specifying a product requirement, an agent needs to navigate multiple types of webpages and issue diverse actions to find, customize, and purchase an item. WebShop provides several challenges for language grounding including understanding compositional instructions, query (re-)formulation, comprehending and acting on...
- [139] [Prime saliency in semantic priming with 18-month-olds](#)
Nicolas Gillen, ..., and Kim Plunkett. Cognition, 2024. 39 citations.
0% Topic Match
No summary or abstract available
- [140] [Empirical results](#)
Niels Geiger and Vadim Kufenko. Business Cycles and Economic Crises, 2018. 21 citations.
0% Topic Match
No summary or abstract available
- [141] [Program Synthesis with Large Language Models](#)
Jacob Austin, ..., and Charles Sutton. ArXiv, 2021. 2655 citations.
Not measured Topic Match
Abstract: This paper explores the limits of the current generation of large language models for program synthesis in general purpose programming languages. We evaluate a collection of such models (with between 244M and 137B parameters) on two new benchmarks, MBPP and MathQA-Python, in both the few-shot and fine-tuning regimes. Our benchmarks are designed to measure the ability of these models to synthesize short Python programs from natural language descriptions. The Mostly Basic Programming Problems (MBPP) dataset contains 974 programming tasks, designed to be solvable by entry-level programmers. The MathQA-Python dataset, a Python version of the MathQA benchmark, contains 23914 problems that...
- [142] [A survey on large language model based autonomous agents](#)
Lei Wang, ..., and Ji-rong Wen. Frontiers of Computer Science, 2023. 1897 citations.
Not measured Topic Match
Abstract: Autonomous agents have long been a research focus in academic and industry communities. Previous research often focuses on training agents with limited knowledge within isolated environments, which diverges significantly from human learning processes, and makes the agents hard to achieve human-like decisions. Recently, through the acquisition of vast amounts of Web knowledge, large language models (LLMs) have shown potential in human-level intelligence, leading to a surge in research on LLM-based autonomous agents. In this paper, we present a comprehensive survey of these studies, delivering a systematic review of LLM-based autonomous agents from a holistic perspective. We first discuss the construction...
- [143] [Self-Refine: Iterative Refinement with Self-Feedback](#)
Aman Madaan, ..., and Peter Clark. ArXiv, 2023. 2367 citations.
Not measured Topic Match
Abstract: Like humans, large language models (LLMs) do not always generate the best output on their first try. Motivated by how humans refine their written text, we introduce Self-Refine, an approach for improving initial outputs from LLMs through iterative feedback and refinement. The main idea is to generate an initial output using an LLMs; then, the same LLMs provides feedback for its output and uses it to refine itself, iteratively. Self-Refine does not require any supervised training data, additional training, or reinforcement learning, and instead uses a single LLM as the generator, refiner, and feedback provider. We evaluate Self-Refine across 7...
- [144] [Unraveling the Potential of Large Language Models in Code Translation: How Far are We?](#)
Qingxiao Tao, ..., and Beijun Shen. 2024 31st Asia-Pacific Software Engineering Conference (APSEC), 2024. 21 citations.
Not measured Topic Match
Abstract: While large language models (LLMs) exhibit state-of-the-art performance in various tasks, recent studies have revealed their struggle for code translation. This is because they haven't been extensively pre-trained with parallel multilingual code, which code translation heavily depends on. Moreover, existing benchmarks only cover a limited subset of common programming languages, and thus cannot reflect the full potential of LLMs in code translation. In this paper, we conduct a large-scale empirical study to exploit the capabilities and incapacities of LLMs in code translation tasks. We first craft a novel benchmark called PolyHumanEval by extending HumanEval to a multilingual benchmark of 14...
- [145] [Can LLMs Replace Manual Annotation of Software Engineering Artifacts?](#)

Toufique Ahmed, ..., and Michael Pradel. 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), 2024. 38 citations.

Not measured Topic Match

Abstract: Experimental evaluations of software engineering innovations, e.g., tools and processes, often include human-subject studies as a component of a multi-pronged strategy to obtain greater generalizability of the findings. However, human-subject studies in our field are challenging, due to the cost and difficulty of finding and employing suitable subjects, ideally, professional programmers with varying degrees of experience. Meanwhile, large language models (LLMs) have recently started to demonstrate human-level performance in several areas. This paper explores the possibility of substituting costly human subjects with much cheaper LLM queries in evaluations of code and coderelated artifacts. We study this idea by applying six...

[146] [Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key?](#)

Qineng Wang, ..., and Yangqiu Song. Unknown journal, 2024. 122 citations.

Not measured Topic Match

Abstract: Recent progress in LLMs discussion suggests that multi-agent discussion improves the reasoning abilities of LLMs. In this work, we reevaluate this claim through systematic experiments, where we propose a novel group discussion framework to enrich the set of discussion mechanisms. Interestingly, our results show that a single-agent LLM with strong prompts can achieve almost the same performance as the best existing discussion approach on a wide range of reasoning tasks and backbone LLMs. We observe that the multi-agent discussion performs better than a single agent only when there is no demonstration in the prompt. Further study reveals the common interaction...

[147] [Large Language Models for Test-Free Fault Localization](#)

A. Z. Yang, ..., and Vincent J. Hellendoorn. 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE), 2023. 144 citations.

Not measured Topic Match

Abstract: Fault Localization (FL) aims to automatically localize buggy lines of code, a key first step in many manual and automatic debugging tasks. Previous FL techniques assume the provision of input tests, and often require extensive program analysis, program instrumentation, or data preprocessing. Prior work on deep learning for APR struggles to learn from small datasets and produces limited results on real-world programs. Inspired by the ability of large language models (LLMs) of code to adapt to new tasks based on very few examples, we investigate the applicability of LLMs to line level fault localization. Specifically, we propose to overcome the...

[148] [Towards an understanding of large language models in software engineering tasks](#)

Zibin Zheng, ..., and Weicheng Wang. Empirical Software Engineering, 2023. 124 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have drawn widespread attention and research due to their astounding performance in text generation and reasoning tasks. Derivative products, like ChatGPT, have been extensively deployed and highly sought after. Meanwhile, the evaluation and optimization of LLMs in software engineering tasks, such as code generation, have become a research focus. However, there is still a lack of systematic research on applying and evaluating LLMs in software engineering. Therefore, this paper comprehensively investigate and collate the research and products combining LLMs with software engineering, aiming to answer two questions: (1) What are the current integrations of LLMs with...

[149] [Self-Organized Agents: A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization](#)

Yoichi Ishibashi and Yoshimasa Nishimura. ArXiv, 2024. 66 citations.

Not measured Topic Match

Abstract: Recent advancements in automatic code generation using large language model (LLM) agent have brought us closer to the future of automated software development. However, existing single-agent approaches face limitations in generating and improving large-scale, complex codebases due to constraints in context length. To tackle this challenge, we propose Self-Organized multi-Agent framework (SoA), a novel multi-agent framework that enables the scalable and efficient generation and optimization of large-scale code. In SoA, self-organized agents operate independently to generate and modify code components while seamlessly collaborating to construct the overall codebase. A key feature of our framework is the automatic multiplication of agents...

[150] [SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents](#)

Feng Lin, ..., and Tse-Hsun Chen. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2024. 37 citations.

Not measured Topic Match

Abstract: Software process models are essential to facilitate collaboration and communication among software teams to solve complex development tasks. Inspired by these software engineering practices, we present FlowGen - a code generation framework that emulates software process models based on multiple Large Language Model (LLM) agents. We emulate three process models, FlowGenWaterfall, FlowGenTDD, and FlowGenScrum, by assigning LLM agents to embody roles (i.e., requirement engineer, architect, developer, tester, and scrum master) that correspond to everyday development activities and organize their communication patterns. The agents work collaboratively using chain-of-thought and prompt composition with continuous selfrefinement to improve the code quality. We use...

[151] [Automated Cloud Infrastructure-as-Code Reconciliation with AI Agents](#)

Zhenning Yang, ..., and Ang Chen. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Cloud infrastructure is managed through a mix of interfaces -- traditionally, cloud consoles, command-line interfaces (CLI), and SDKs are the tools of choice. Recently, Infrastructure-as-Code/IaC frameworks (e.g., Terraform) have quickly gained popularity. Unlike conventional tools, IaC-frameworks encode the infrastructure in a "source-of-truth" configuration. They are capable of automatically carrying out modifications to the cloud -- deploying, updating, or destroying resources -- to bring the actual infrastructure into alignment with the IaC configuration. However, when IaC is used alongside consoles, CLIs, or SDKs, it loses visibility into external changes, causing infrastructure drift, where the configuration becomes outdated, and later IaC operations may undo...

[152] [TREAT: A Code LLMs Trustworthiness / Reliability Evaluation and Testing Framework](#)

Shuzheng Gao, ..., and Michael R. Lyu. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Large language models are fundamentally transforming the software engineering landscape, demonstrating exceptional capabilities across diverse tasks such as code generation, debugging, and testing. Despite this rapid progress, a significant gap remains in how to comprehensively evaluate these models' trustworthiness in real-world software engineering scenarios. Existing benchmarks suffer from limited task scope and fail to incorporate critical evaluation aspects such as the robustness and reliability of models. To bridge this gap, we present an evaluation framework called TREAT (Code LLMs Trustworthiness / Reliability Evaluation And Testing) that provides a holistic assessment of model performance in code intelligence tasks. Our evaluation framework addresses...

[153] [InfCode-C++: Intent-Guided Semantic Retrieval and AST-Structured Search for C++ Issue Resolution](#)

Qingao Dong, ..., and Weifeng Lv. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Large language model (LLM) agents have recently shown strong performance on repository-level issue resolution, but existing systems are almost exclusively designed for Python and rely heavily on lexical retrieval and shallow code navigation. These approaches transfer poorly to C++ projects, where overloaded identifiers, nested namespaces, template instantiations, and deep control-flow structures make context retrieval and fault localization substantially more difficult. As a result, state-of-the-art Python-oriented agents show a drastic performance drop on the C++ subset of MultiSWE-bench. We introduce INFCode-C++, the first C++-aware autonomous system for end-to-end issue resolution. The system combines two complementary retrieval mechanisms -- semantic code-intent retrieval...

[154] [SkyRL-Agent: Efficient RL Training for Multi-turn LLM Agent](#)

Shiyi Cao, ..., and Ion Stoica. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: We introduce SkyRL-Agent, a framework for efficient, multi-turn, long-horizon agent training and evaluation. It provides efficient asynchronous dispatching, lightweight tool integration, and flexible backend interoperability, enabling seamless use with existing RL frameworks such as SkyRL-train, VeRL, and Tinker. Using SkyRL-Agent, we train SA-SWE-32B, a software engineering agent trained from Qwen3-32B (24.4% Pass@1) purely with reinforcement learning. We introduce two key components: an optimized asynchronous pipeline dispatcher that achieves a 1.55x speedup over naive asynchronous batching, and a tool-enhanced training recipe leveraging an AST-based search tool to facilitate code navigation, boost rollout Pass@K, and improve training efficiency. Together, these optimizations enable...

[155] [Scalable Supervising Software Agents with Patch Reasoner](#)

Junjielong Xu, ..., and Pinjia He. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: While large language model agents have advanced software engineering tasks, the unscalable nature of existing test-based supervision is limiting the potential improvement of data scaling. The reason is twofold: (1) building and running test sandbox is rather heavy and fragile, and (2) data with high-coverage tests is naturally rare and threatened by test hacking via edge cases. In this paper, we propose R4P, a patch verifier model to provide scalable rewards for training and testing SWE agents via reasoning. We consider that patch verification is fundamentally a reasoning task, mirroring how human repository maintainers review patches without writing and running...

[156] [TheMCPCompany: Creating General-purpose Agents with Task-specific Tools](#)

Reza Esfandiarpour, ..., and Anthony Aue. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Since the introduction of the Model Context Protocol (MCP), the number of available tools for Large Language Models (LLMs) has increased significantly. These task-specific tool sets offer an alternative to general-purpose tools such as web browsers, while being easier to develop and maintain than GUIs. However, current general-purpose agents predominantly rely on web browsers for interacting with the environment. Here, we introduce TheMCPCompany, a benchmark for evaluating tool-calling agents on tasks that involve interacting with various real-world services. We use the REST APIs of these services to create MCP servers, which include over 18,000 tools. We also provide manually annotated...

[157] [BugPilot: Complex Bug Generation for Efficient Learning of SWE Skills](#)

Atharv Sonwane, ..., and Xingdi Yuan. ArXiv, 2025. 2 citations.

Not measured Topic Match

Abstract: High quality bugs are key to training the next generation of language model based software engineering (SWE) agents. We introduce a novel method for synthetic generation of difficult and diverse bugs. Our method instructs SWE Agents to introduce a feature into the codebase whereby they may unintentionally break tests, resulting in bugs. Prior approaches often induce an out-of-distribution effect by generating bugs intentionally (e.g. by introducing local perturbation to existing code), which does not reflect realistic development processes. We perform qualitative analysis to demonstrate that our approach for generating bugs more closely reflects the patterns found in human-authored edits. Through...

[158] [LLM Agents for Automated Web Vulnerability Reproduction: Are We There Yet?](#)

Bin Liu, ..., and Haoyu Wang. ArXiv, 2025. 1 citations.

Not measured Topic Match

Abstract: Large language model (LLM) agents have demonstrated remarkable capabilities in software engineering and cybersecurity tasks, including code generation, vulnerability discovery, and automated testing. One critical but underexplored application is automated web vulnerability reproduction, which transforms vulnerability reports into working exploits. Although recent advances suggest promising potential, challenges remain in applying LLM agents to real-world web vulnerability reproduction scenarios. In this paper, we present the first comprehensive evaluation of state-of-the-art LLM agents for automated web vulnerability reproduction. We systematically assess 20 agents from software engineering, cybersecurity, and general domains across 16 dimensions, including technical capabilities, environment adaptability, and user experience factors...

[159] [REFINE: Enhancing Program Repair Agents through Context-Aware Patch Refinement](#)

Anvith Pabba, ..., and Baishakhi Ray. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) have recently shown strong potential in automatic program repair (APR), especially in repository-level settings where the goal is to generate patches based on natural language issue descriptions, large codebases, and regression tests. However, despite their promise, current LLM-based APR techniques often struggle to produce correct fixes due to limited understanding of code context and over-reliance on incomplete test suites. As a result, they frequently generate Draft Patches-partially correct patches that either incompletely address the bug or overfit to the test cases. In this work, we propose a novel patch refinement framework, Refine, that systematically transforms Draft...

[160] [Tailored Primitive Initialization is the Secret Key to Reinforcement Learning](#)

Yi-Fan Yao, ..., and Chuang Gan. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: Reinforcement learning (RL) has emerged as a powerful paradigm for enhancing the reasoning capabilities of large language models (LLMs). While RL has demonstrated substantial performance gains, it still faces key challenges, including low sampling efficiency and a strong dependence on model initialization: some models achieve rapid improvements with minimal RL steps, while others require significant training data to make progress. In this work, we investigate these challenges through the lens of reasoning token coverage and argue that initializing LLMs with diverse, high-quality reasoning primitives is essential for achieving stable and sample-efficient RL training. We propose Tailor, a finetuning pipeline that...

[161] [Red Teaming Program Repair Agents: When Correct Patches can Hide Vulnerabilities](#)

Simin Chen, ..., and Baishakhi Ray. ArXiv, 2025. 0 citations.

Not measured Topic Match

Abstract: LLM-based agents are increasingly deployed for software maintenance tasks such as automated program repair (APR). APR agents automatically fetch GitHub issues and use backend LLMs to generate patches that fix the reported bugs. However, existing work primarily focuses on the functional correctness of APR-generated patches, whether they pass hidden or regression tests, while largely ignoring potential security risks. Given the openness of platforms like GitHub, where any user can raise issues and participate in discussions, an important question arises: Can an adversarial user submit a valid issue on GitHub that misleads an LLM-based agent into generating a functionally correct but...

[162] [Leveraging Large Language Model for Information Retrieval-based Bug Localization](#)

Moumita Asad, ..., and Sam Malek. ArXiv, 2025. 1 citations.

Not measured Topic Match

Abstract: Information Retrieval-based Bug Localization (IRBL) aims to identify buggy source files for a given bug report. Traditional and deep-learning-based IRBL techniques often suffer from vocabulary mismatch and dependence on project-specific metadata, while recent Large Language Model (LLM)-based approaches are limited by insufficient contextual information. To address these issues, we propose GenLoc, an LLM-based technique that combines semantic retrieval with code-exploration functions to iteratively analyze the code base and identify potential buggy files. We evaluate GenLoc on two diverse datasets: a benchmark of 9,097 bugs from six large open-source projects and the GHRB (GitHub Recent Bugs) dataset of 131 recent bugs...

[163] [Exploring the Relationship Between Code Metrics and the Ability of Large Language Models to Solve Code Issues](#)

Oliver Ramirez and Péter Eklér. 2025 IEEE 16th International Conference on Cognitive Infocommunications (CogInfoCom), 2025. 0 citations.

Not measured Topic Match

Abstract: Large Language Models (LLMs) are increasingly integrated into software engineering workflows, assisting with tasks such as code generation, unit testing, and bug resolution. Despite their growing use, limited research has examined how code-level metrics such as Halstead complexity, maintainability index, and source lines of code correlate with issue resolution rates for LLM-powered agents when modifying existing codebases. The paper bridges the well established field of code metrics with the emerging domain of AI software engineering agents by exploring correlations between these metrics and the success of LLM driven agents in resolving GitHub issues. The key contribution is a quantitative analysis...